Data Warehousing and Online Analytical Processing

Contents

- Data Warehouse;
- Data Warehouse Modeling: Schema and Measures;
- OLAP Operations;
- Data Cube Computation;
- Data Cube Computation Methods

Data Warehouse

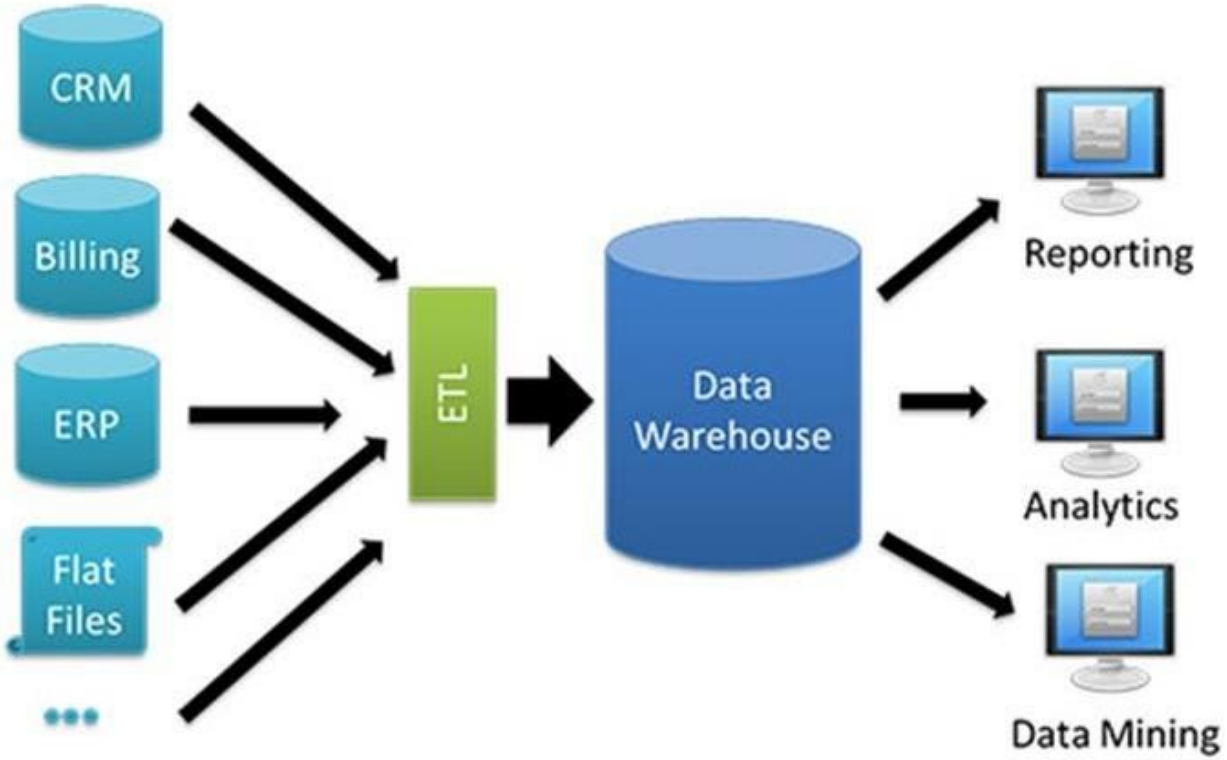
- Data warehouse is an information system that contains historical and commutative data from single or multiple sources. It simplifies reporting and analysis process of the organization. It is also a single version of truth for any company for decision making and forecasting.

- A data warehouse is constructed by integrating data from multiple heterogeneous sources that support analytical reporting, structured and/or ad hoc queries, and decision making.

- A data warehouse is a subject-oriented, integrated, time-variant and non-volatile collection of data in support of management's decision-making process.

- **Data warehousing** is the process of constructing and using a data warehouse. Data warehousing involves data cleaning, data integration, and data consolidations.

Architecture of Data Warehouse



ETL (Extract, Transform, and Load) Process

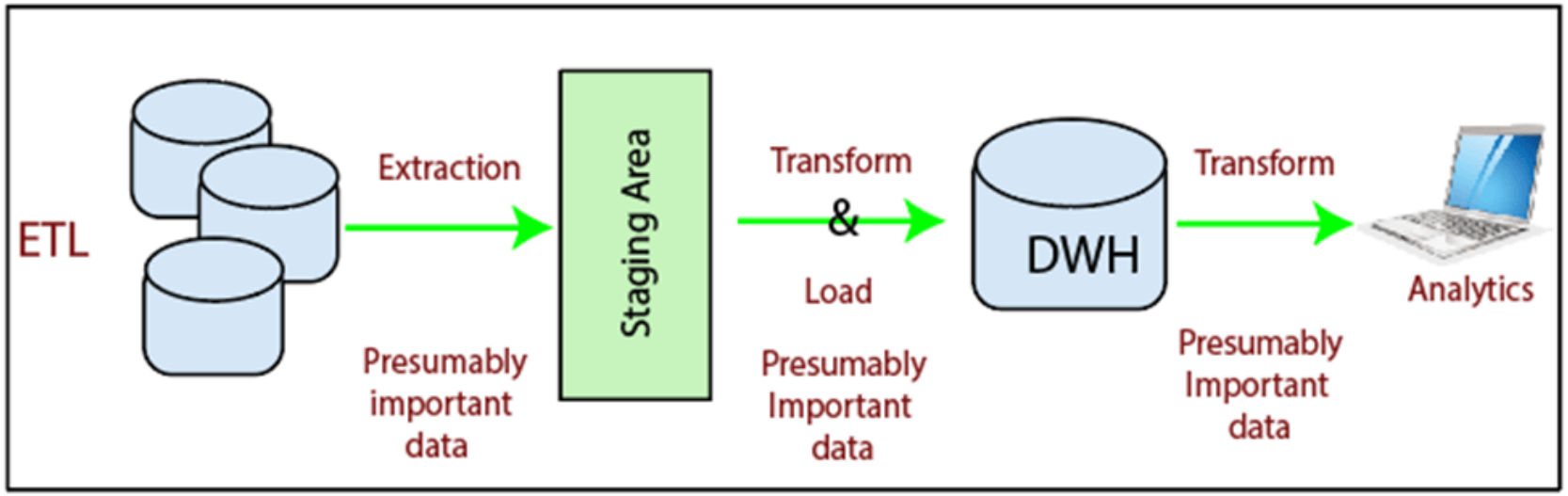


Fig: ETL Process

Note: Staging area is the place where data is preprocessed such as cleaning of data

- The mechanism of extracting information from source systems and bringing it into the data warehouse is commonly called ETL, which stands for Extraction, Transformation and Loading.
- ETL is a recurring method (daily, weekly, monthly) of a Data warehouse system and needs to be agile, automated, and well documented.
- ETL consists of three separate phases:
  - Extraction
  - Transformation
  - Loading

Extraction

- First stage of the ETL- Extraction is the operation of extracting information from a source system for further use in a data warehouse environment.
- Extraction process is often one of the most time-consuming tasks in the ETL.

- The source systems might be complicated and poorly documented, and thus determining which data needs to be extracted can be difficult.
- The data has to be extracted several times in a periodic manner to supply all changed data to the warehouse and keep it up-to-date.

Cleaning

- The cleaning stage is crucial in a data warehouse technique because it is supposed to improve data quality.
- The primary data cleaning features found in ETL tools are rectification and homogenization.
- They use specific dictionaries to rectify typing mistakes and to recognize synonyms, as well as rule-based cleansing to enforce domain-specific rules and defines appropriate associations between values.

Transformation

- It converts records from its operational source format into a particular data warehouse format.
- Following are the main transformation processes aimed at populating the reconciled data layer:
    - Conversion and normalization that operate on both storage formats and units of measure to make data uniform.
    - Matching that associates equivalent fields in different sources.
    - Selection that reduces the number of source fields and records.
- Cleansing and Transformation processes are often closely linked in ETL tools.

Loading

- Loading is the process of writing the data into the target database.
- During the loading step, it is necessary to ensure that the load is performed correctly and with as little resources as possible.

- Loading can be carried in two ways:

- **Refresh:** Data Warehouse data is completely rewritten. This means that older file is replaced. Refresh is usually used in combination with static extraction to populate a data warehouse initially.

- **Update:** Only those changes applied to source information are added to the Data Warehouse. An update is typically carried out without deleting or modifying preexisting data. This method is used in combination with incremental extraction to update data warehouses regularly.

Characteristics of Data Warehouse

- **Subject-Oriented**: A data warehouse can be used to analyze a particular subject area. These subjects can be sales, marketing, distributions, etc.
- **Integrated**: A data warehouse integrates data from multiple data sources. A data warehouse is developed by integrating data from varied sources like a mainframe, relational databases, flat files, etc.
- For example, source A and source B may have different ways of identifying a product, but in a data warehouse, there will be only a single way of identifying a product.
- **Time-Variant**: Historical data is kept in a data warehouse. For example, one can retrieve data from 3 months, 6 months, 12 months, or even older data from a data warehouse.
- For example, a transaction system may hold the most recent address of a customer, where a data warehouse can hold all addresses associated with a customer.
- **Non-volatile**: Once data is in the data warehouse, it will not change. So, historical data in a data warehouse should never be altered.

Metadata Repository

- **Meta data** is the data defining warehouse objects. It stores: Description of the structure of the data warehouse
  - schema, view, dimensions, hierarchies, derived data definition, data mart locations and contents
- Operational meta-data
  - data lineage (history of migrated data and transformation path), currency of data (active, archived, or purged), monitoring information (warehouse usage statistics, error reports, audit trails)
- The algorithms used for summarization
- The mapping from operational environment to the data warehouse
- Data related to system performance
  - warehouse schema, view and derived data definitions
- Business data
  - business terms and definitions, ownership of data, charging policies

Differences between Operational Database Systems and Data Warehouses

- The **Operational Database** is the source of information for the data warehouse. It includes detailed information used to run the day to day operations of the business. The data frequently changes as updates are made and reflect the current value of the last transactions.
- **Operational Database Systems** also called as OLTP (Online Transactions Processing), are used to manage dynamic data in real-time.
- **Data Warehouse Systems** serve users or knowledge workers in the purpose of data analysis and decision-making. Such systems can organize and present information in specific formats to accommodate the diverse needs of various users. These systems are called as Online-Analytical Processing (OLAP) Systems.
- Data Warehouse and the OLTP database are both relational databases. However, the goals of both these databases are different.

The major distinguishing features of OLTP and OLAP are summarized as follows:

- **Users and system orientation:** An OLTP system is *customer-oriented* and is used for transaction and query processing by clerks, clients, and information technology professionals. An OLAP system is *market-oriented* and is used for data analysis by knowledge workers, including managers, executives, and analysts.
- **Data contents:** An OLTP system manages current data that, typically, are too detailed to be easily used for decision making. An OLAP system manages large amounts of historic data, provides facilities for summarization and aggregation, and stores and manages information at different levels of granularity.
- **Database design:** An OLTP system usually adopts an entity-relationship (ER) data model and an application-oriented database design. An OLAP system typically adopts either a *star* or a *snowflake* model and a subject-oriented database design.

- **View:** An OLTP system focuses mainly on the current data within an enterprise or department, without referring to historic data or data in different organizations. In contrast, an OLAP system often spans multiple versions of a database schema, due to the evolutionary process of an organization. OLAP systems also deal with information that originates from different organizations, integrating information from many data stores.
- **Access patterns:** The access patterns of an OLTP system consist mainly of short, atomic transactions. Such a system requires concurrency control and recovery mechanisms. However, accesses to OLAP systems are mostly read-only operations (because most data warehouses store historic rather than up-to-date information), although many could be complex queries.

Table :Comparison of OLTP and OLAP Systems

Feature	*OLTP*	*OLAP*
Characteristic	operational processing	informational processing
Orientation	transaction	analysis
User	clerk, DBA, database professional	knowledge worker (e.g., manager, executive, analyst)
Function	day-to-day operations	long-term informational requirements decision support
DB design	ER-based, application-oriented	star/snowflake, subject-oriented
Data	current, guaranteed up-to-date	historic, accuracy maintainedover time
Summarization	primitive, highly detailed	summarized, consolidated
View	detailed, flat relational	summarized, multidimensional
Unit of work	short, simple transaction	complex query
Access	read/write	mostly read
Focus	data in	information out

Operations	**index/hash on primary key**	**lots of scans**
Number of records accessed	**tens**	**millions**
Number of users	**thousands**	**hundreds**
DB size	**GB to high-order GB**	**≥ TB**
Priority	**high performance, high availability**	**high flexibility, end-user autonomy**
Metric	**transaction throughput**	**query throughput, response time**

Data cube: a multidimensional data model

- A multidimensional model views data in the form of a data-cube. A data cube enables data to be modeled and viewed in multiple dimensions. It is defined by **dimensions** and **facts**.
- The dimensions are the perspectives or entities concerning which an organization keeps records.

- For example, for subject sales in a company, the possible perspectives may include time, item, branch, and location. Those perspectives are modeled as **dimensions**. In the simplest multidimensional data model, a dimension table can be built for each dimension. For example, a dimension table for *item* may contain the attributes *item_name*, *brand* and *type*.
- Each dimension has a table related to it, called a **dimensional table**, which describes the dimension further. For example, a dimensional table for an item may contain the attributes item_name, brand, and type.

- **Facts**, example, for subject sales in a company, the facts may be *dollars_sold* (sales amount in dollars), *units_sold* (number of units sold), and *amount_budgeted*. Facts are typically numerical, but still may take some other data types, such as categorical data or text.

- In a data warehouse, a **fact table** stores the names of the *facts*, or measures, as well as (foreign) keys referencing to each of the related dimension tables.

Table 3.1 2-D view of sales data according to *time* and *item*.

	location = "Vancouver"			
time (quarter)	**item (type)**			
	home entertainment	**computer**	**phone**	**security**
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Note: The sales are from branches located in the city of Vancouver. The measure displayed is dollars_sold *(in thousands).*

Table 3.2 3-D view of sales data according to *time*, *item*, and *location*.

	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
time	item				item				item				item			
	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Note: The measure displayed is dollars_sold *(in thousands).*

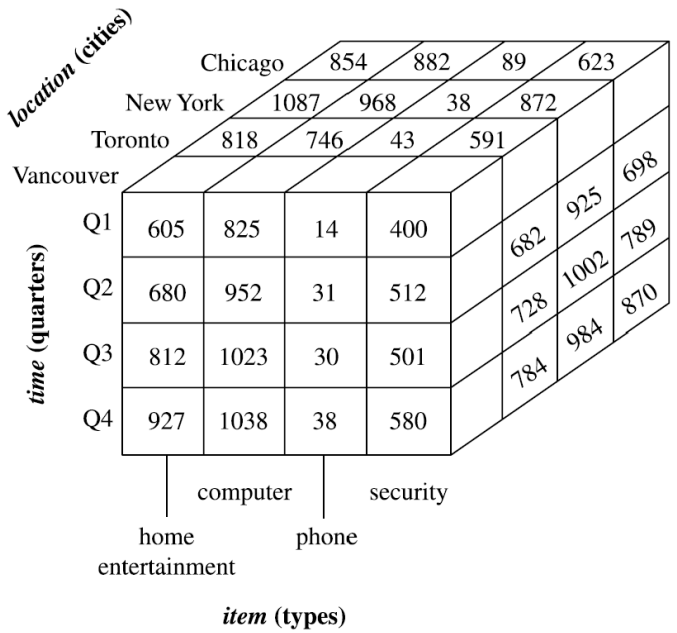


Fig: A 3-D data cube representation of the data in Table 3.2, according to *time*, *item*, and *location*. The measure displayed is *dollars_sold* (in thousands).

- Online Analytical Processing Server (OLAP) is based on the multidimensional data model.
- It allows managers, and analysts to get an insight of the information through fast, consistent, and interactive access to information.

OLAP Operations

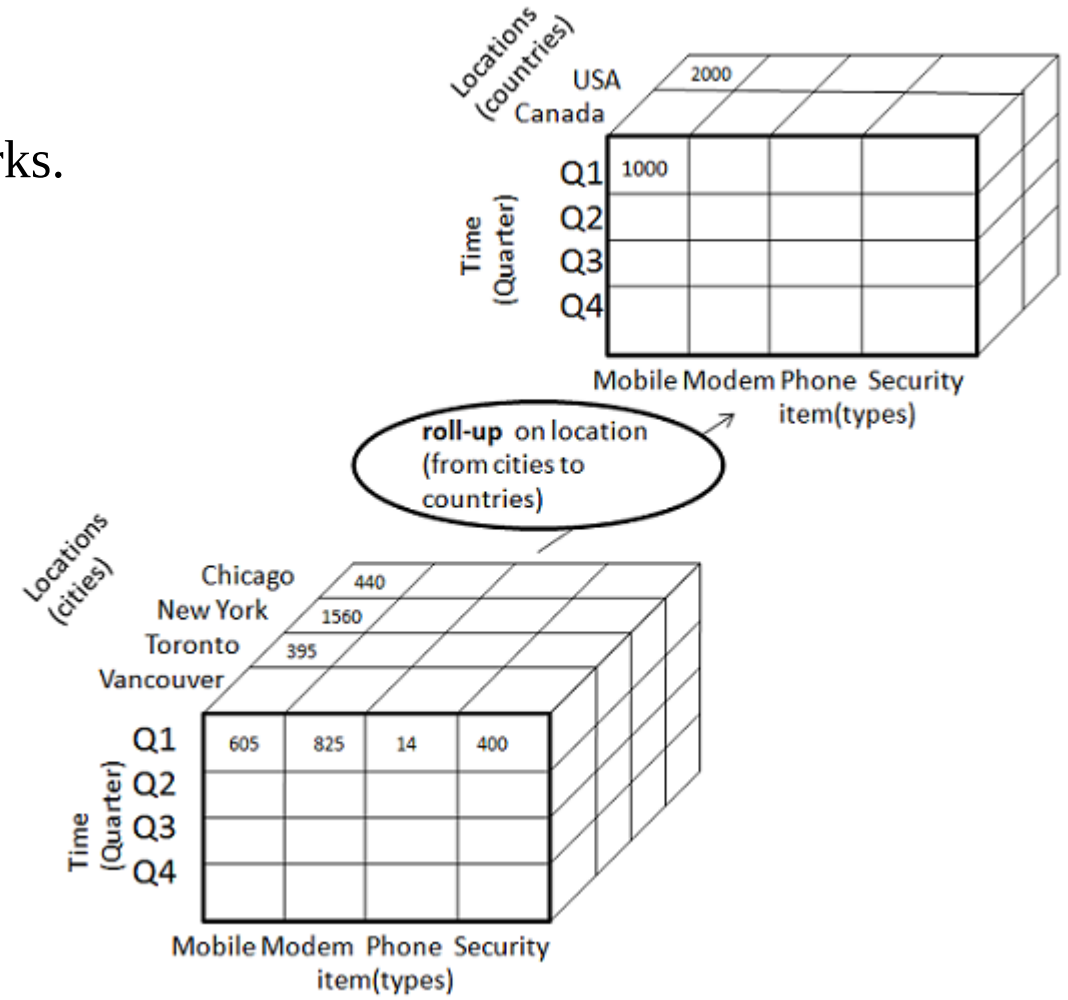
1. Roll-up (Drill-up)
2. Drill-down (Roll-down)
3. Slice
4. Dice
5. Pivot (rotate)

Roll-up:

Roll-up performs aggregation on a data cube in any of the following ways −

- By climbing up a concept hierarchy for a dimension
- By dimension reduction

The following diagram illustrates how roll-up works.



- Roll-up is performed by climbing up a concept hierarchy for the dimension location.
- Initially the concept hierarchy was "street < city < province < country". On rolling up, the data is aggregated by ascending the location hierarchy from the level of city to the level of country.
- The data is grouped into cities rather than countries.
- When roll-up is performed, one or more dimensions from the data cube are removed.

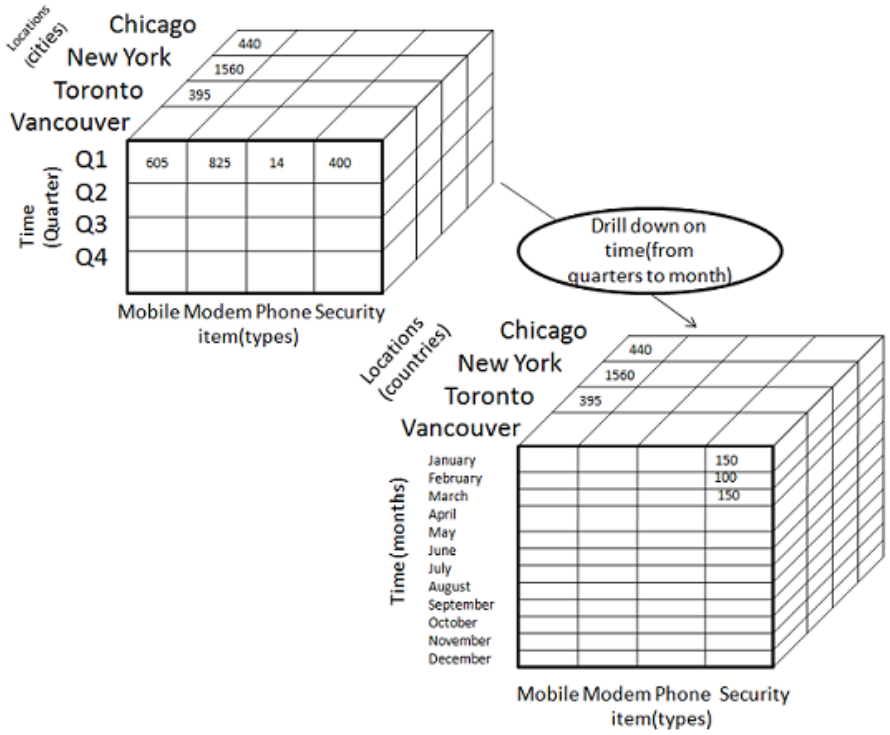
Drill-down:

Drill-down is the reverse operation of roll-up. It is performed by either of the following ways−

- By stepping down a concept hierarchy for a dimension
- By introducing a new dimension.

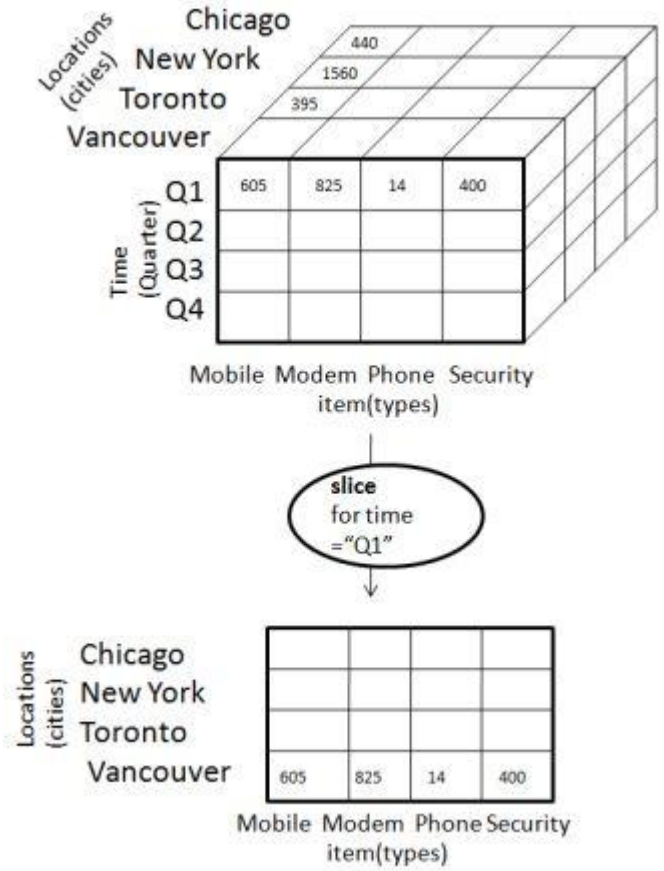
The following diagram illustrates how drill-down works −

- Drill-down is performed by stepping down a concept hierarchy for the dimension time.
- Initially the concept hierarchy was “day < month < quarter<year.” On drilling down, the time dimension is descended from the level of quarter to the level of month.
- When drill-down is performed, one or more dimensions from the data cube are added.
- It navigates the data from less detailed data to highly detailed data.



Slice:

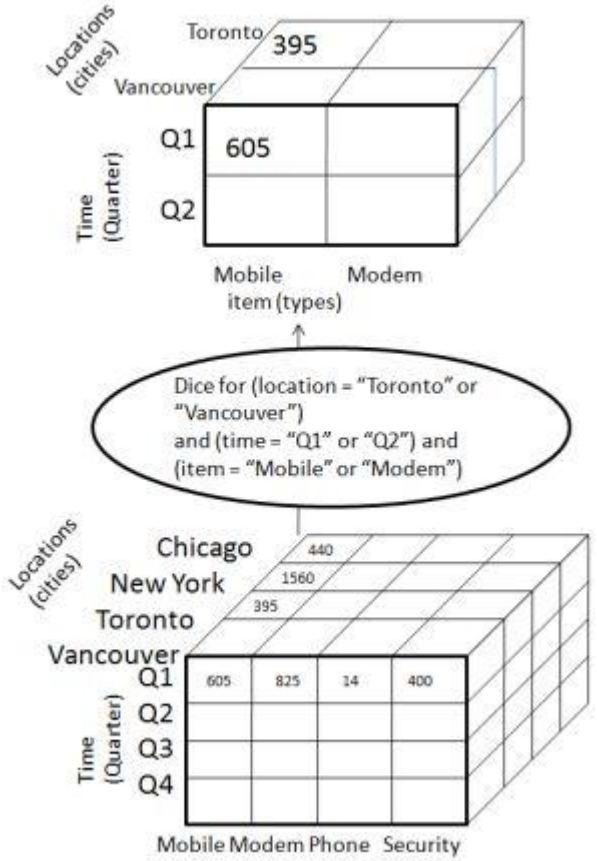
The slice operation selects one particular dimension from a given cube and provides a new sub-cube. Consider the following diagram that shows how slice works.



- Here Slice is performed for the dimension "time" using the criterion time = "Q1".
- It will form a new sub-cube by selecting one or more dimensions.

Dice:

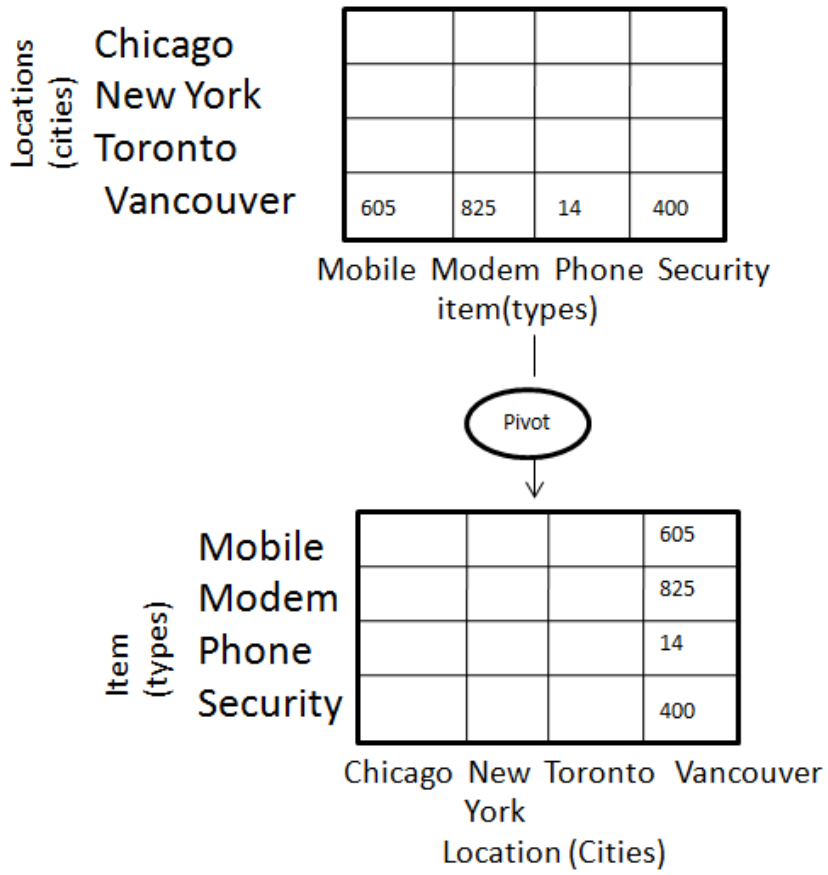
Dice selects two or more dimensions from a given cube and provides a new sub-cube. Consider the following diagram that shows the dice operation.



- The dice operation on the cube based on the following selection criteria involves three dimensions.
  - (location = "Toronto" or "Vancouver")
  - (time = "Q1" or "Q2")
  - (item =" Mobile" or "Modem")

Pivot:

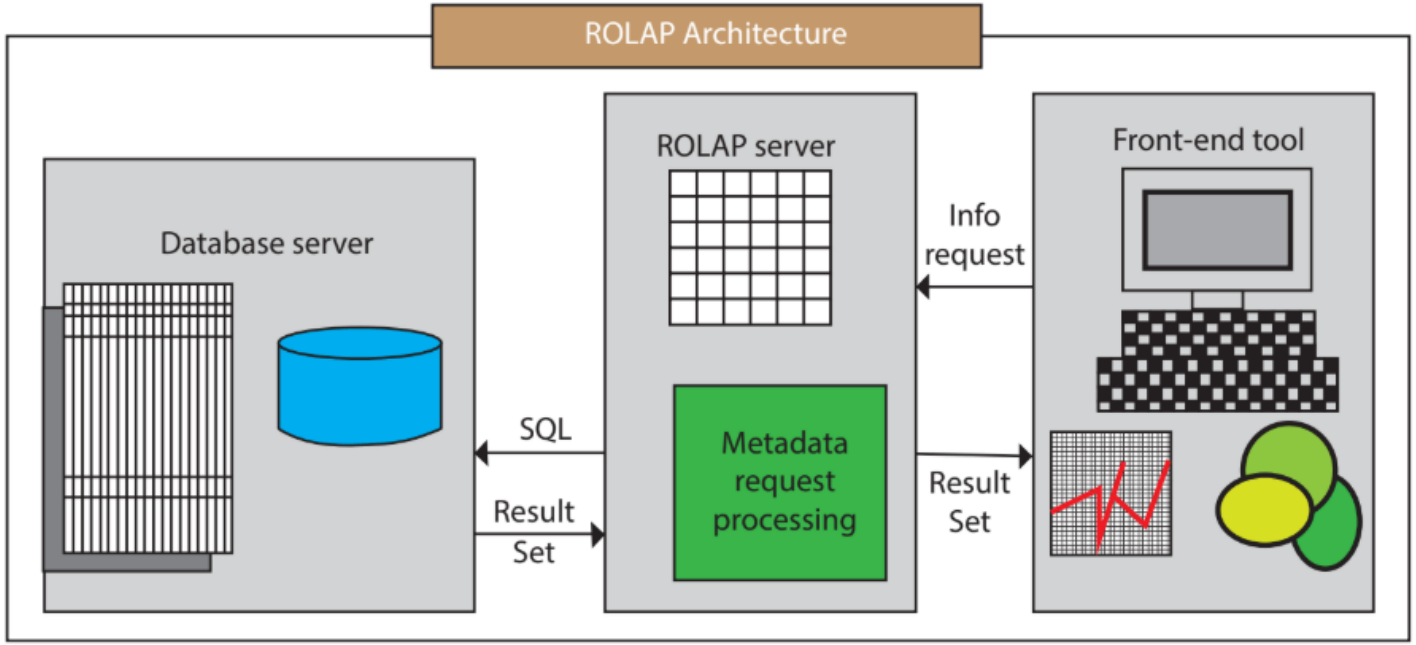
The pivot operation is also known as rotation. It rotates the data axes in view in order to provide an alternative presentation of data. Consider the following diagram that shows the pivot operation.



Types of OLAP Server

- **ROLAP** (Relational OLAP), an application based on relational DBMSs.
- **MOLAP** (Multidimensional OLAP), an application based on multidimensional DBMSs.
- **HOLAP** (Hybrid OLAP), an application using both relational and multidimensional techniques.

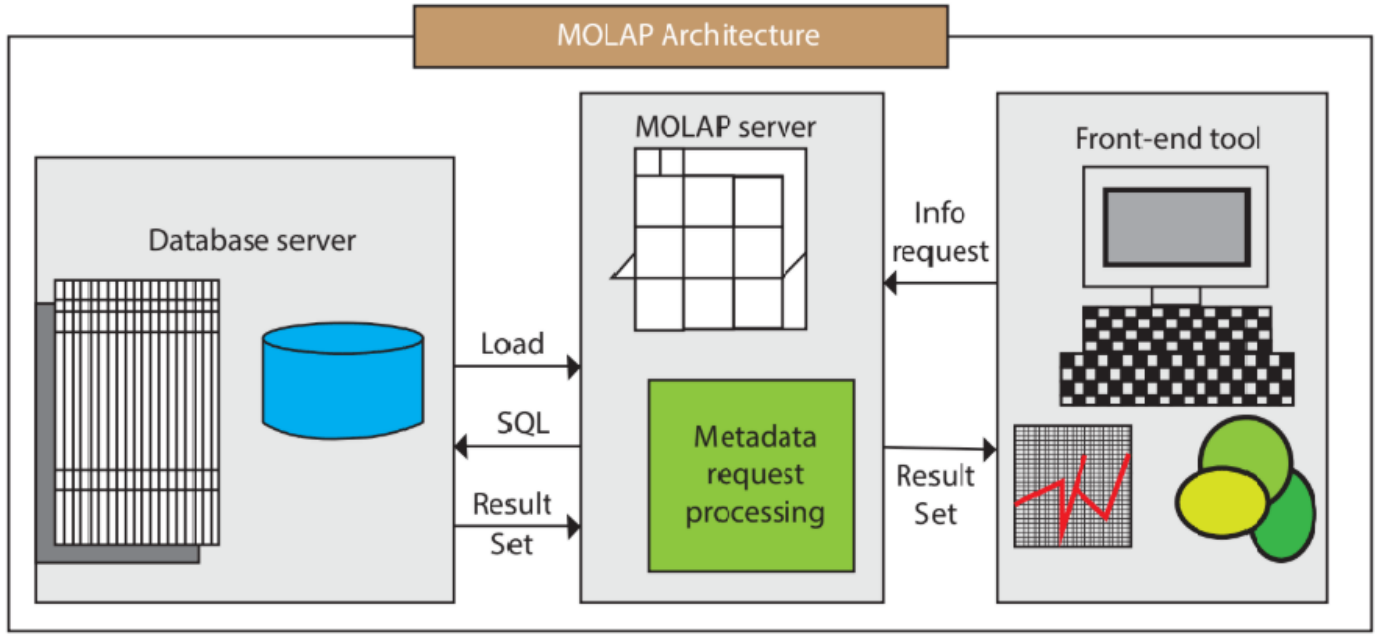
Relational OLAP (ROLAP) Server:



- They use a relational or extended-relational DBMS to save and handle warehouse data, and OLAP middleware to provide missing pieces.

- ROLAP servers contain optimization for each DBMS back end, implementation of aggregation navigation logic, and additional tools and services.
- ROLAP Architecture includes the following components : Database server, ROLAP server, Front-end tool.

Multidimensional OLAP (MOLAP) Server:

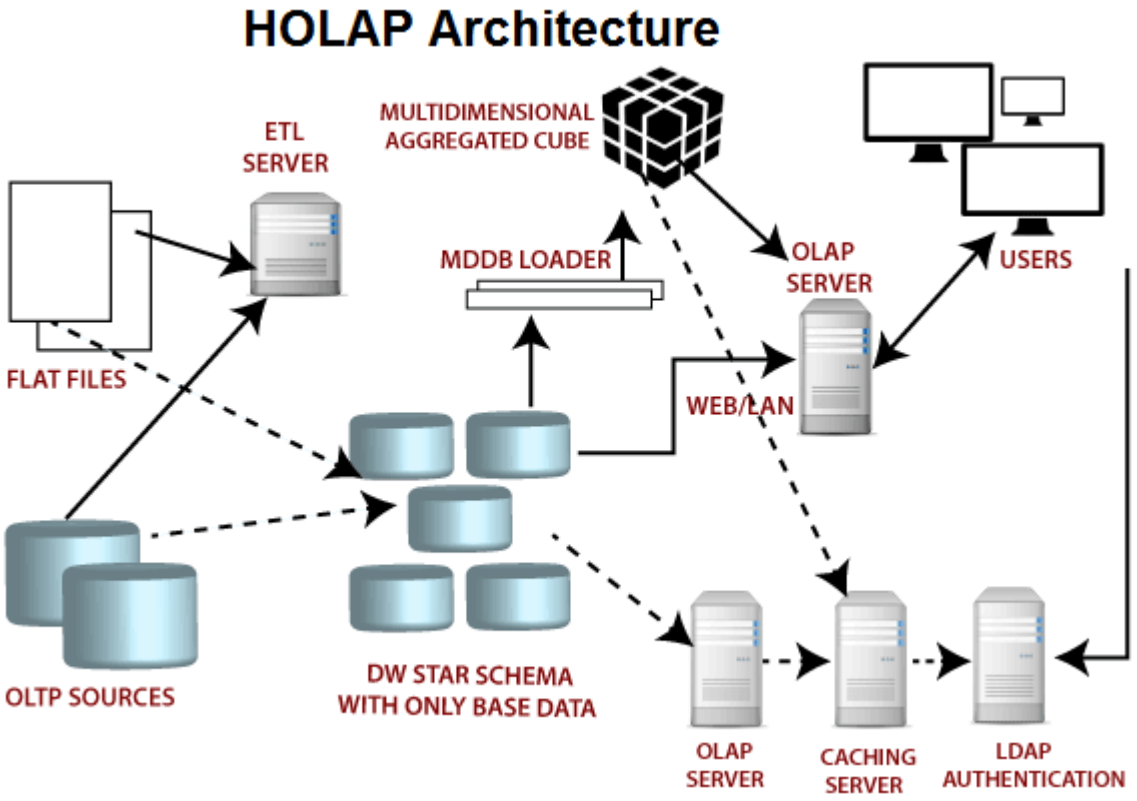


- These servers support multidimensional data views through *array-based multidimensional storage engines*. They map multidimensional views directly to data cube array structures.

- Many MOLAP servers adopt a two-level storage representation to **handle dense and sparse data sets**:
- Denser sub-cubes are identified and stored as array structures, whereas sparse sub-cubes employ compression technology for efficient storage utilization.
- MOLAP Architecture includes the following components: Database server, MOLAP server, Front-end tool.

Hybrid OLAP (HOLAP) Server:

- The hybrid OLAP approach combines ROLAP and MOLAP technology, benefiting from the greater scalability of ROLAP and the faster computation of MOLAP. For example, a HOLAP server may allow large volumes.



Schemas for multidimensional data models: stars, snowflakes, and fact constellations

- A **Data warehouse conceptual data model** is nothing but a highest-level relationships between the different entities (in other word different table) in the data model.
- Schema is a logical description of the entire database. It includes the name and description of records of all record types including all associated data-items and aggregates. Much like a database, a data warehouse also requires to maintain a schema.
- A database uses relational model, while a data warehouse uses **Star, Snowflake, and Fact Constellation schema :** Schemas for Multidimensional Data Models.

Star Schema:

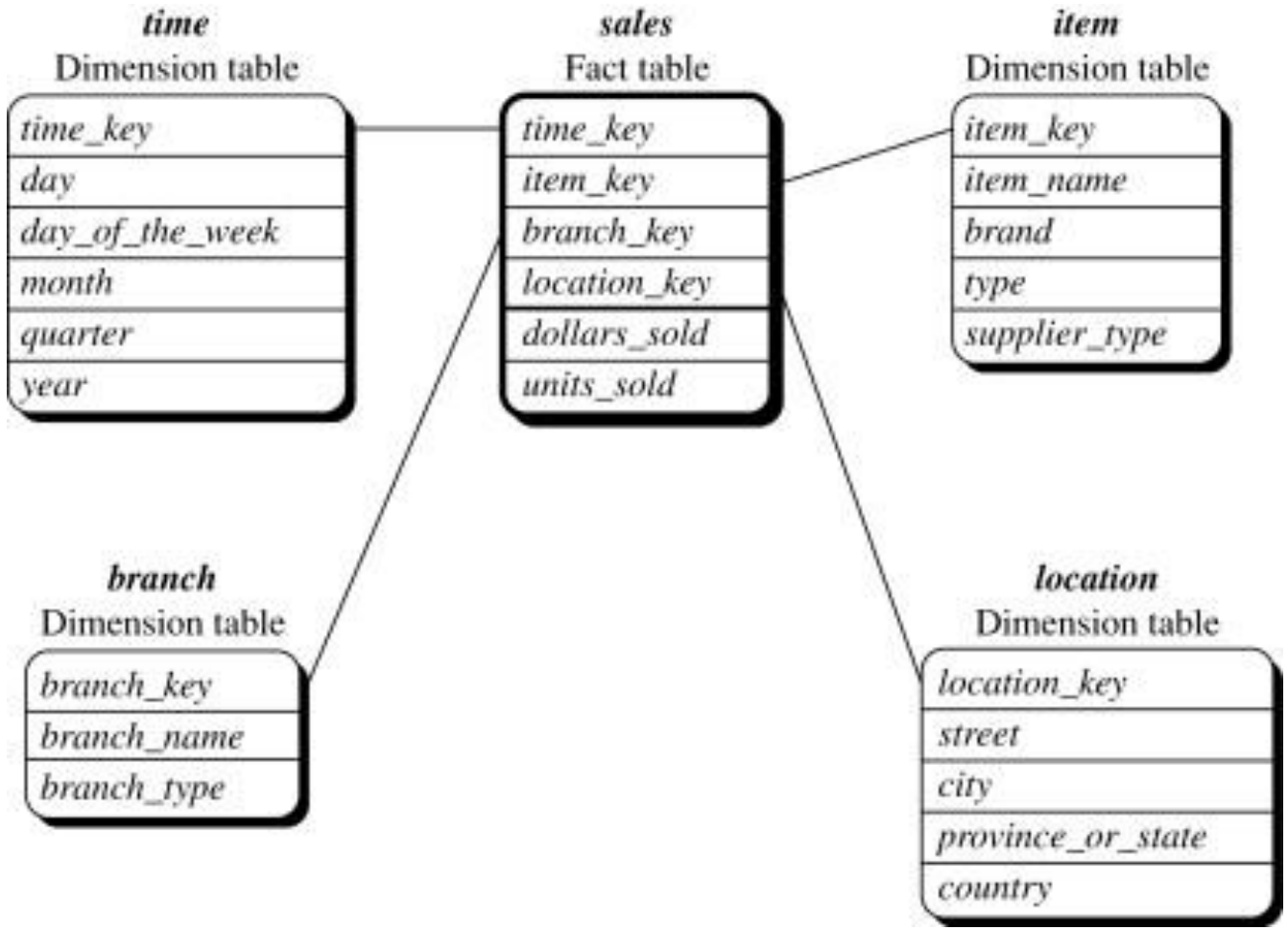


Figure : Star schema of *sales* data warehouse.

- Each dimension in a star schema is represented with only one-dimension table-(which contains set of attributes).
- The following diagram shows the sales data of a company with respect to the four dimensions, namely time, item, branch, and location.
- The schema contains a central fact table for *sales* : *dollars_sold* and *units_sold*. To minimize the size of the fact table, dimension identifiers (e.g., *time_key* and *item_key*) are system-generated identifiers.
- Notice that in the star schema, each dimension is represented by only one table, and each table contains a set of attributes.
- For example, the *location* dimension table contains the attribute set {*location_key, street, city, province_or_state, country* }.
- Notice that in the star schema, each dimension is represented by only one table, and each table contains a set of attributes.

- For example, the *location* dimension table contains the attribute set {*location_key, street, city, province_or_state, country* }.

Snowflake Schema:

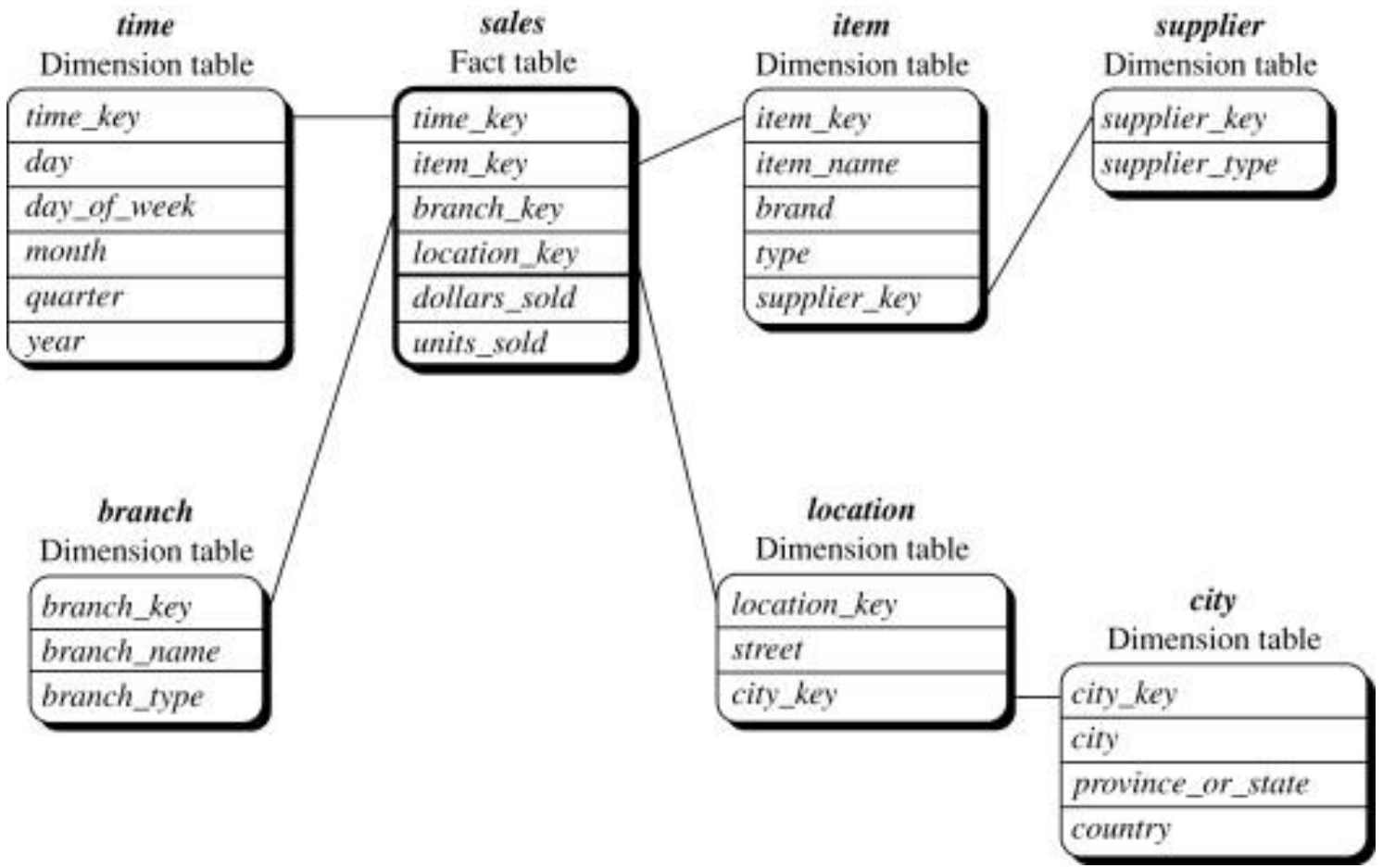


Figure : Snowflake schema of a *sales* data warehouse.

- The snowflake schema is a variant of the star schema model, where some dimension tables are *normalized*, thereby further splitting the data into additional tables. The resulting schema graph forms a shape similar to a snowflake.
- The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form to reduce redundancies. Such a table is easy to maintain and saves storage space.
- However, this space savings is negligible in comparison to the typical magnitude of the fact table.
- Unlike Star schema, the dimensions table in a snowflake schema are normalized.
- For example, the item dimension table in star schema is normalized and split into two dimension tables, namely item and supplier table.

- For example, the *item* dimension table now contains the attributes *item_key*, *item_name*, *brand*, *type*, and *supplier_key*, where *supplier_key* is linked to the *supplier* dimension table, containing *supplier_key* and *supplier_type* information.
- Similarly, the single dimension table for *location* in the star schema can be normalized into two new tables: *location* and *city*. The *city_key* in the new *location* table links to the *city* dimension.

Fact Constellation/ galaxy Schema:

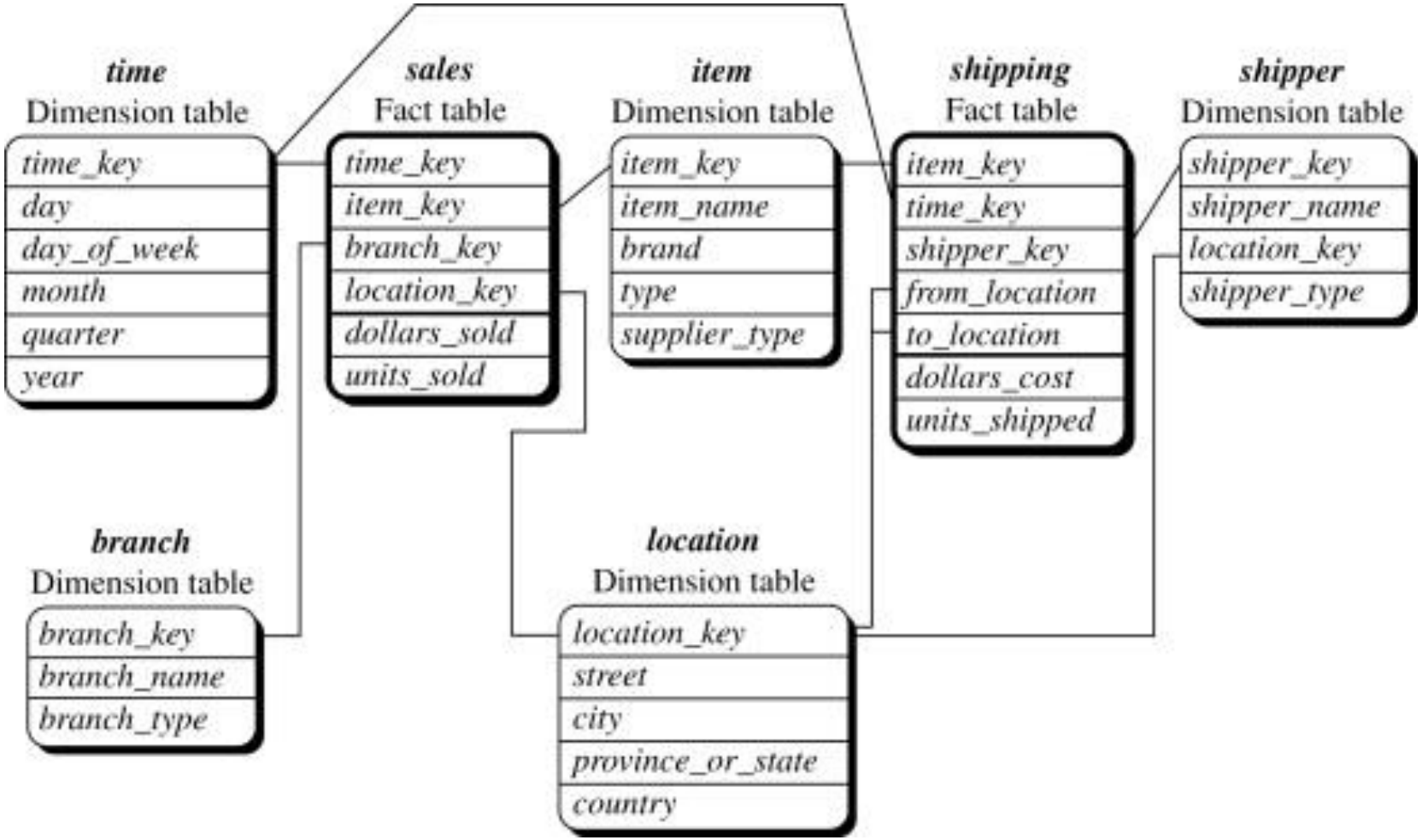


Figure : Fact constellation schema of a sales and shipping data warehouse.

- Fact constellation: Sophisticated applications may require multiple fact tables to *share* dimension tables.
- This kind of schema can be viewed as a collection of stars, and hence is called a **galaxy schema or a fact constellation.**
- The following diagram shows two fact tables, namely *sales* and *shipping*.
- The sales fact table is same as that in the star schema.
- The *sales* table definition is identical to that of the star schema .
- The *shipping* table has five dimensions, or keys—*item_key, time_key, shipper_key, from_location*, and *to_location*—and two measures—*dollars_cost* and *units_shipped*.
- A fact constellation schema allows dimension tables to be shared between fact tables.
- For example, the dimensions tables for *time*, *item*, and *location* are shared between the *sales* and *shipping* fact tables.

Measures: categorization and computation

- A **measure** in a data cube is a numeric function that can be evaluated at each point in the data cube space. A measure value is computed for a given point by aggregating the data corresponding to the respective dimension–value pairs defining the given point.
- For example, the measure *total-sales* for the cell *time* = “Q1,” *location* = “Vancouver,” *item* = “computer” is computed by summing up all the amounts happened in Q1, at the branch of Vancouver, and about computers from the fact table.
- Measures can be organized into three categories—**distributive, algebraic, and holistic**—based on the kind of aggregate functions used.

Distributive:

- An aggregate function is *distributive* if it can be computed in a distributed manner as follows.
- Suppose the data is partitioned into *n* sets arbitrarily. We apply the aggregate function to each partition, resulting in *n* aggregate values. If the result derived by applying the function to the *n* aggregate values is the same as that derived by applying the function to the entire data set (i.e., without partitioning), the function is said to be computed in a distributed manner.
- For example, sum() can be computed for a data cube by first partitioning the cube into a set of subcubes, computing sum() for each subcube, and then summing up the counts obtained for each subcube. Hence sum() is a distributive aggregate function. For the same reason, count(), min(), and max() are distributive aggregate functions.

Algebraic:

- An aggregate function is *algebraic* if it can be computed by an algebraic function with *M* arguments (where *M* is a fixed positive integer), each of which is obtained by applying a distributive aggregate function.
- For example, avg() (average) can be computed by sum()/count() with two arguments, where both sum() and count() are distributive aggregate functions.
- Similarly, it can be shown that min_N() and max_N() (which find the *N* minimum and *N* maximum values, respectively, in a given set) and standard_deviation() are algebraic aggregate functions.
- A measure is *algebraic* if it is obtained by applying an algebraic aggregate function.

Holistic:

- An aggregate function is *holistic* if there is no constant bound on the storage size needed to describe a subaggregate. i.e., there does not exist an algebraic function with *M* arguments (where *M* is a constant) that characterizes the computation.
- Some examples of holistic functions include median(), mode(), and rank(). A measure is *holistic* if it is obtained by applying a holistic aggregate function.

Data Warehouse Architectures

Single-tier architecture

- The objective of a single layer is to minimize the amount of data stored. This goal is to remove data redundancy. This architecture is not frequently used in practice.

Two-tier architecture

- Two-layer architecture separates physically available sources and data warehouse. This architecture is not expandable and also not supporting a large number of end-users. It also has connectivity problems because of network limitations.

Three-tier architecture

- This is the most widely used architecture.: It consists of the Top, Middle and Bottom Tier.

Three-tier data warehouse architecture

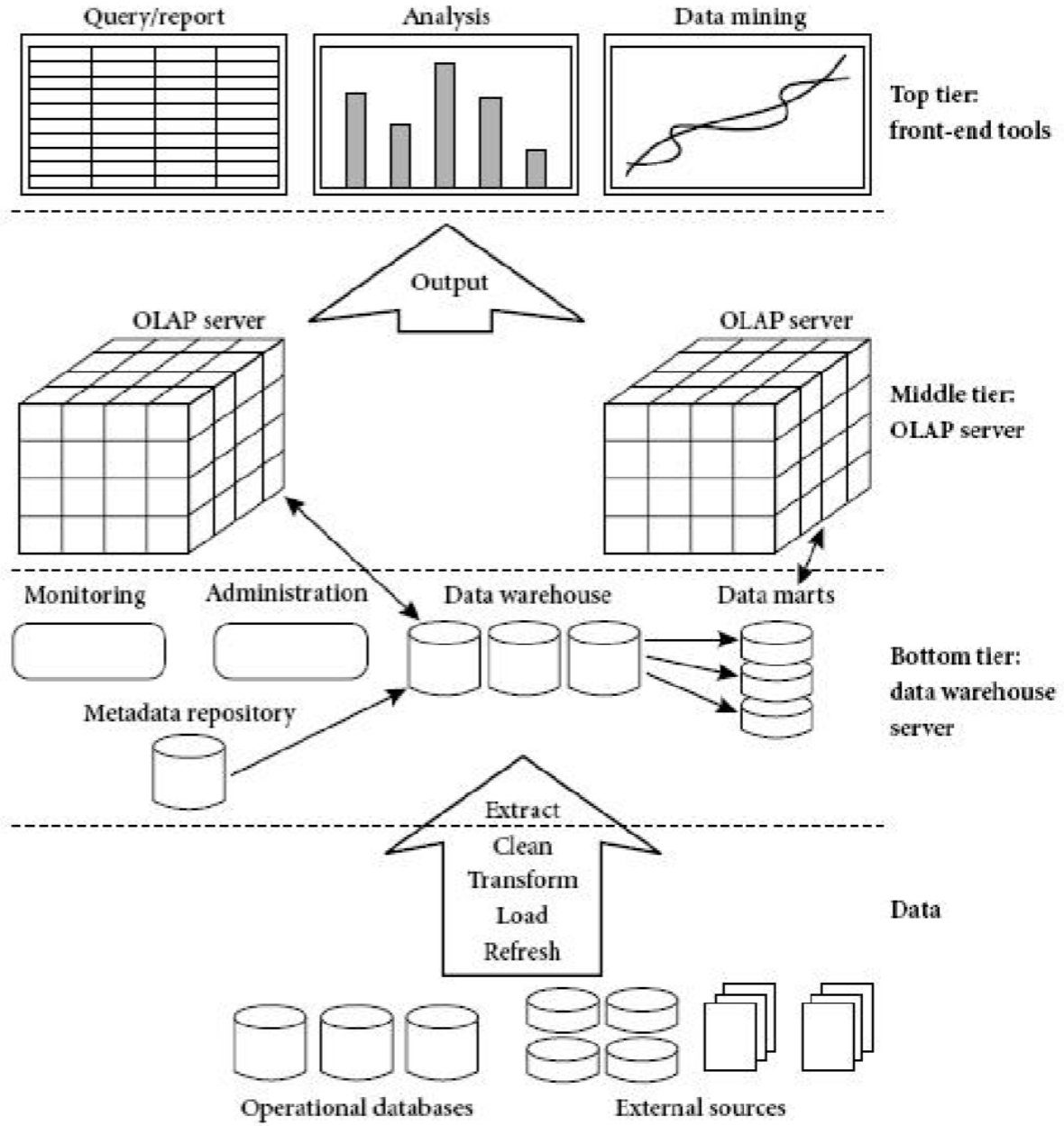


Figure : A three-tier data warehousing architecture.

A Three Tier Data Warehouse Architecture:

1. The **bottom tier** is a warehouse database server that is almost always a relational database system. Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources (e.g., customer profile information provided by external consultants).

- These tools and utilities perform data extraction, cleaning, and transformation (e.g., to merge similar data from different sources into a unified format), as well as load and refresh functions to update the data warehouse.
- The data are extracted using API known as gateways. A gateway is supported by the underlying DBMS and allows client programs to generate SQL code to be executed at a server.

- Examples of gateways include ODBC (Open Database Connection) and OLEDB (Object Linking and Embedding Database) by Microsoft and JDBC (Java Database Connection).
- This tier also contains a metadata repository, which stores information about the data warehouse and its contents. The metadata repository is further described in.

2. The **middle tier** is an OLAP server that is typically implemented using either,

- Relational OLAP (ROLAP) model (i.e., an extended relational DBMS that maps operations on multidimensional data to standard relational operations); or
- Multidimensional OLAP (MOLAP) model (i.e., a special-purpose server that directly implements multidimensional data and operations).

3. The **top tier** is a front-end client layer, which contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

Data Warehouse Models

From the perspective of data warehouse architecture, we have the following data warehouse models

- Enterprise Warehouse
- Data mart
- Virtual Warehouse

Enterprise warehouse:

- An enterprise warehouse collects all of the information about subjects spanning the entire organization.
- It provides corporate-wide data integration, usually from one or more operational systems or external information providers, and is cross-functional in scope.
- It typically contains detailed data as well as summarized data, and can range in size from a few gigabytes to hundreds of gigabytes, terabytes, or beyond.

- An enterprise data warehouse may be implemented on traditional mainframes, computer superservers, or parallel architecture platforms. It requires extensive business modeling and may take years to design and build.

Data Mart

- Data mart contains a subset of organization-wide data. This subset of data is valuable to specific groups of an organization.
- In other words, we can claim that data marts contain data specific to a particular group. For example, the marketing data mart may contain data related to items, customers, and sales. Data marts are confined to subjects.

- Window-based or Unix/Linux-based servers are used to implement data marts. They are implemented on low-cost servers.
- The implementation data mart cycles is measured in short periods of time, i.e., in weeks rather than months or years .
- The life cycle of a data mart may be complex in long run, if its planning and design are not organization-wide.
- Data marts are small in size.
- Data marts are customized by department.
- The source of a data mart is departmentally structured data warehouse.

Virtual warehouse:

- A virtual warehouse is a set of views over operational databases. It is easy to build a virtual warehouse.
- A virtual warehouse is easy to build but requires excess capacity on operational database servers.

Data Warehouse Implementation

- Data warehouses contain huge volumes of data.
- OLAP servers demand that decision support queries be answered in the order of seconds.
- Data warehouse systems to support highly efficient,
  1. **cube computation techniques,**
  2. **access methods, and**
  3. **query processing techniques.**

1. Efficient Computation of Data Cubes

- At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions.
- In SQL terms, these aggregations are referred to as group-by's. Each group-by can be represented by a cuboid, where the set of group-by's forms a lattice of cuboids defining a data cube.
- We explore issues relating to the efficient computation of data cubes.

The compute cube Operator

- One approach to cube computation extends SQL so as to include a compute cube operator.
- The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation. This can require excessive storage space, especially for large numbers of dimensions.

- **Example :** A data cube is a lattice of cuboids. Suppose that you want to create a data cube for *AllElectronics* sales that contains the following: *city*, *item*, *year*, and *sales in dollars*.

- You want to be able to analyze the data, with queries such as the following:
  - "Compute the sum of sales, grouping by city and item."
  - "Compute the sum of sales, grouping by city."
  - "Compute the sum of sales, grouping by item."

- What is the total number of cuboids, or group-by's, that can be computed for this data cube? Taking the three attributes, *city, item*, and *year*, as the dimensions for the data cube, and *sales in dollars* as the measure, the total number of cuboids, or groupby's, that can be computed for this data cube is $2^3=8$.

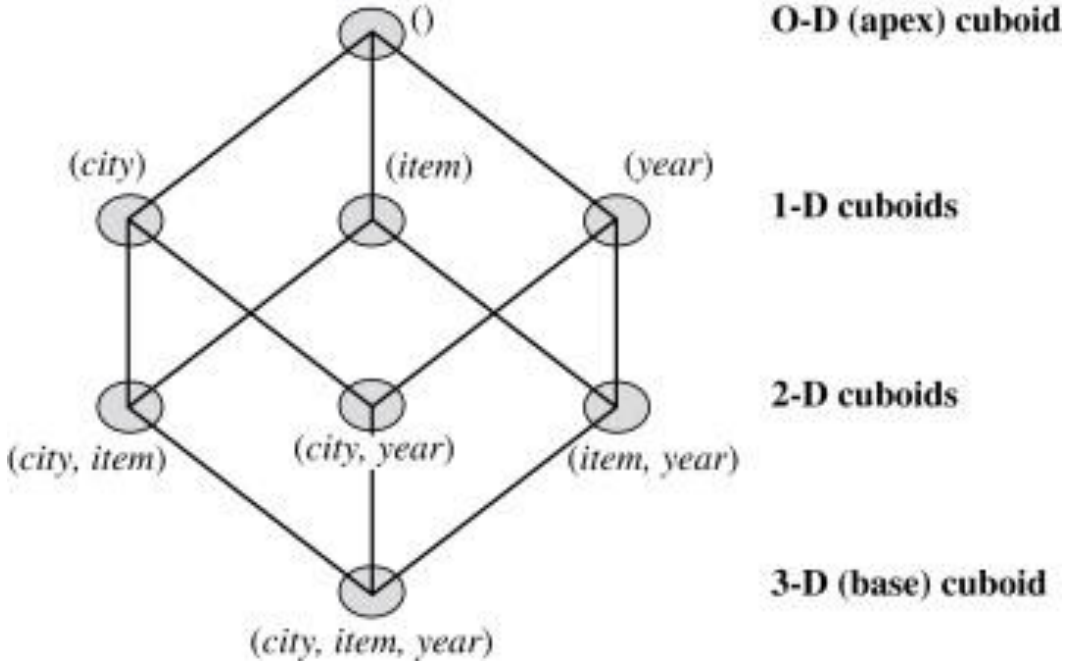


Figure : Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by.

The base cuboid contains *city, item,* and *year* dimensions.

- For an n-dimensional data cube, the total number of cuboids that can be generated (including the cuboids generated by climbing up the hierarchies along each dimension) is

Total number of cuboids=$\prod_{i=1}^{n}(li + 1)$, where L_i is the number of levels associated with dimension i.

Partial Materialization : **data cube computation**

Example : Sales Data Cube: Suppose, company has a data cube with the following 3 dimensions:

- **Time** (Year, Quarter, Month)
- **Location** (Country, State, City)
- **Product** (Category, Brand, Item)

No Materialization

- Only the base cuboid is stored.
- The system stores only the base data (transactions table).
- To answer a query like "total sales by year and product category", the system must compute this from scratch every time.

Full Materialization

- All possible cuboids in the cube lattice are precomputed and stored.
- All possible aggregations are precomputed: year-wise totals, city-level totals, brand-level, combinations like (Time, Location), (Time, Product), etc.
- For 3 dimensions, there are 2^3 =8 cuboids;

Partial Materialization

- Only a carefully selected subset of cuboids is precomputed.
  - Most frequently used aggregations (e.g., sales by country and year)
  - Cuboids where data volume (tuple count) exceeds a threshold
  - Specific combinations useful for business (e.g., (Product, Time), (Location, Product))

2. Access Methods : two types

- Bitmap Index
- Join Index

Bitmap index

- The **bitmap indexing** method is popular in OLAP products because it allows quick searching in data cubes. The bitmap index is an alternative representation of the *record ID (RID)* list.
- In the bitmap index for a given attribute, there is a distinct bit vector, ***Bv***, for each value *v* in the attribute's domain.
- If a given attribute's domain consists of *n* values, then *n* bits are needed for each entry in the bitmap index (i.e., there are *n* bit vectors).
- If the attribute has the value *v* for a given row in the data table, then the bit representing that value is set to 1 in the corresponding row of the bitmap index. All other bits for that row are set to 0.

Example:

- Figure below shows a base (data) table containing the dimensions *item* and *city*, and its mapping to bitmap index tables for each of the dimensions.

Base table

RID	*item*	*city*
R1	H	V
R2	C	V
R3	P	V
R4	S	V
R5	H	T
R6	C	T
R7	P	T
R8	S	T

item bitmap index table

RID	H	C	P	S
R1	1	0	0	0
R2	0	1	0	0
R3	0	0	1	0
R4	0	0	0	1
R5	1	0	0	0
R6	0	1	0	0
R7	0	0	1	0
R8	0	0	0	1

city bitmap index table

RID	V	T
R1	1	0
R2	1	0
R3	1	0
R4	1	0
R5	0	1
R6	0	1
R7	0	1
R8	0	1

Figure : Indexing OLAP data using bitmap indices.

- The **join indexing** method gained popularity from its use in relational database query processing. Traditional indexing maps the value in a given column to a list of rows having that value. In contrast, join indexing registers the joinable rows of two relations from a relational database.

- For example, if two relations R(RID, A) and S (B, SID) join on the attributes A and B, then the join index record contains the pair (RID, SID)where RID and SID are record identifiers from the R and S relations, respectively.

- Join indexing maintains relationships between attribute values of a dimension (e.g., within a dimension table) and the corresponding rows in the fact table. Join indices may span multiple dimensions to form **composite join indices**.

Example : we defined a star schema for *AllElectronics* of the form "*sales star* [*time*, *item*, *branch*, *location*]: *dollars sold* = sum *(sales in dollars).*"

Solution:

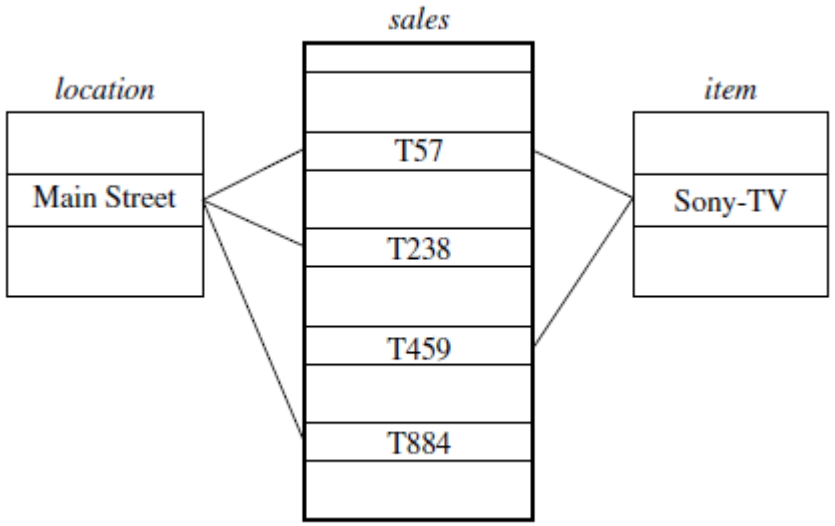


Figure : Linkages between a *sales* fact table and *location* and *item* dimension tables.

Join index table for *location/sales*

location	*sales_key*
. . .	. . .
Main Street	T57
Main Street	T238
Main Street	T884
. . .	. . .

Join index table for *item/sales*

item	*sales_key*
. . .	. . .
Sony-TV	T57
Sony-TV	T459
. . .	. . .

Join index table linking *location* and *item* to *sales*

location	*item*	*sales_key*
. . .	. . .	. . .
Main Street	Sony-TV	T57
. . .	. . .	. . .

Figure : Join index tables based on the linkages between the *sales* fact table and the *location* and *item* dimension tables.

3. Query processing techniques

- The purpose of materializing cuboids and constructing OLAP index structures is to speed up query processing in data cubes.
- Given materialized views, query processing should proceed as follows:

a. **Determine which operations should be performed on the available cuboids:** This involves transforming any selection, projection, roll-up (group-by), and drill-down operations specified in the query into corresponding SQL and/or OLAP operations.

- For example, slicing and dicing a data cube may correspond to selection and/or projection operations on a materialized cuboid.

b. Determine to which materialized cuboid(s) the relevant operations should be applied: This involves identifying all of the materialized cuboids that may potentially be used to answer the query, pruning the set using knowledge of “dominance” relationships among the cuboids, estimating the costs of using the remaining materialized cuboids, and selecting the cuboid with the least cost.

Example :

Suppose that we define a data cube for *AllElectronics* of the form "*sales_cube* [*time, item, location*]: sum*(sales_in_dollars)*."

The dimension hierarchies used are "*day* < *month* < *quarter* < *year*" for *time*; "*item_name* < *brand* < *type*" for *item*; and "*street* < *city* < *province_or_state* < *country*" for *location*.

Suppose that the query to be processed is on (*brand, province or state*), with the selection constant "*year = 2010*.

Solution:

Suppose, that there are four materialized cuboids available, as follows:

cuboid 1: {*year, item name, city*}

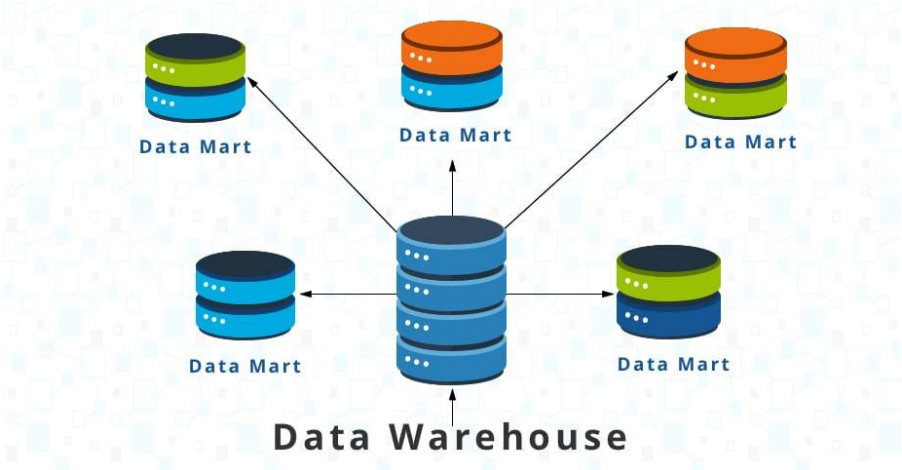
cuboid 2: {*year, brand, country*}

cuboid 3: {*year, brand, province_or_state*}

cuboid 4: {*item_name, province_or_state*}, where *year* = 2010

Data Mart

- A **Data Mart** is focused on a single functional area of an organization and contains a subset of data stored in a Data Warehouse. A Data Mart is a condensed version of Data Warehouse and is designed for use by a specific department, unit or set of users in an organization.
- For examples, Marketing, Sales, HR or finance. It is often controlled by a single department in an organization.



Types of data marts

There are three types of data marts:

a. **Dependent:** A dependent data mart allows you to unite your organization's data in one data warehouse.
b. **Independent:** An independent data mart is a stand-alone system — created without the use of a data warehouse — that focuses on one subject area or business function.
c. **Hybrid:** A hybrid data mart combines data from an existing data warehouse and other operational source systems.

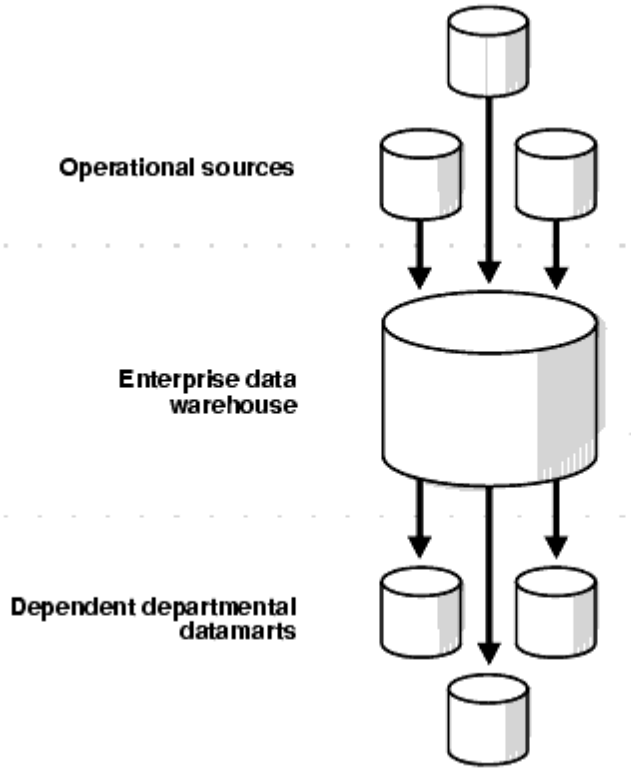


Figure : Dependent Data Mart

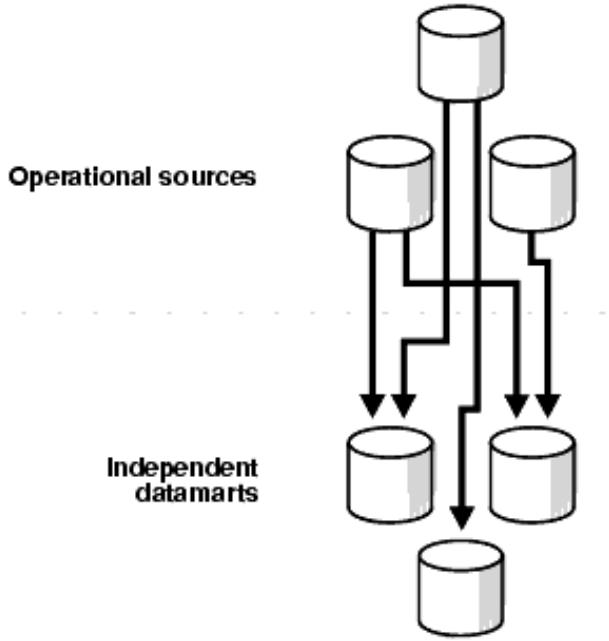


Figure: Independent Data Mart

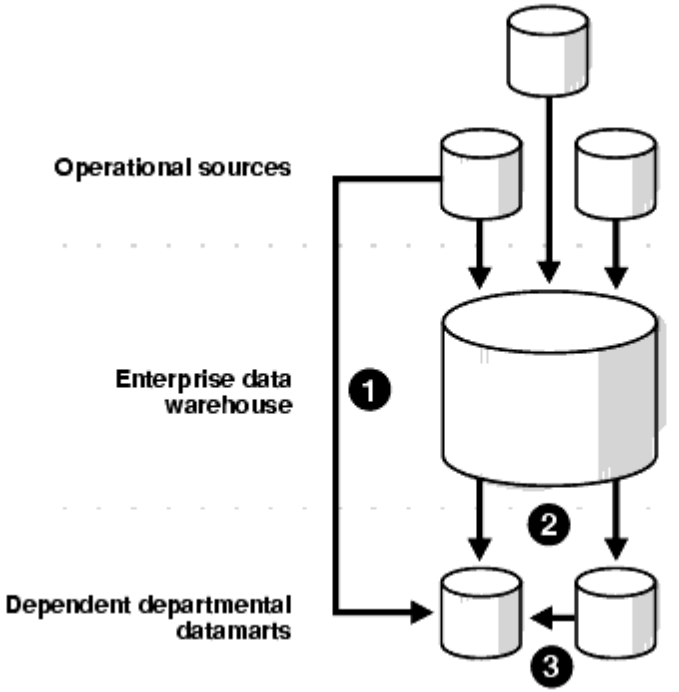


Figure: Hybrid Data Mart

Cube materialization : Full cube, Iceberg cube, Closed cube, Shell cube

- **Full cube:** Full materialization, In order to ensure fast OLAP, it is sometimes desirable to precompute the full cube (i.e., all the cells of all of the cuboids for a given data cube). This, however, is exponential to the number of dimensions. That is, a data cube of n dimensions contains 2^n cuboids. There are even more cuboids if we consider concept hierarchies for each dimension.
- In addition, the size of each cuboid depends on the cardinality of its dimensions. Thus, precomputation of the full cube can require huge and often excessive amounts of memory.
- **Iceberg cube:** Partially materialized, it contains only those cuboid cells whose measures satisfy the aggregate or iceberg condition.
- The aggregate condition could be, minimum support or a lower bound on average, min or max.
- It is called an iceberg cube because it contains only some of the cells of the full cube like the tip of an iceberg.

- The purpose of the Iceberg-cube is to identify and compute only those values that will most likely be required for decision support queries. The aggregate condition specifies which cube values are more meaningful and should therefore be stored. This is one solution to the problem of computing versus storing data cubes.
- An iceberg cube can be specified with an SQL query,

Example:

compute cube *sales iceberg* as

select *month*, *city*, *customer group*, count(*)

from *salesInfo*

cube by *month*, *city*, *customer_group*

having count(*) >= *min_sup*

- The compute cube statement specifies the precomputation of the iceberg cube, *sales iceberg*, with the dimensions *month*, *city*, and *customer group*, and the aggregate measure count(). The input tuples are in the *salesInfo* relation. The cube by clause specifies that aggregates (group-by's) are to be formed for each of the possible subsets of the given dimensions. If we were computing the full cube, each group-by would correspond to a cuboid in the data cube lattice. The constraint specified in the having clause is known as the **iceberg condition**. Here, the iceberg measure is count().
- **Closed cube:** It consists of only closed cells. A cell that has no descendant cell then it is a closed cell.
- A cell, c, is a closed cell if there exists no cell, d, such that d is a specialization (descendant) of cell c (i.e, where d is obtained by replacing a in c with a non- value), and d has the same measure value as c.
- For example, the three cells derived in the preceding paragraph are the three closed cells of the data cube for the data set f.*a*1

- **Shell cube:** Another strategy for partial materialization is to pre-compute only portions and fragments of the cube shell (cuboids involving small no. of dimensions such as 3 to 5), based on the cuboids of interest.
- For example, shell fragment cube ABC contains 7 cuboids: A, B, C, AB, AC, BC, and ABC).

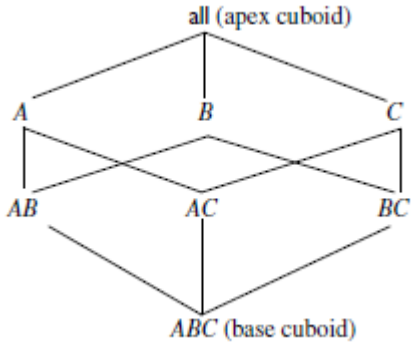


Figure: Lattice of cuboids making up a 3-D data cube with the dimensions *A*, *B*, and *C* for some aggregate measure, *M*

General strategies for cube computation

- In general, there are two basic data structures used for storing cuboids.
- The implementation of relational OLAP (ROLAP) uses relational tables, whereas multidimensional arrays are used in multidimensional OLAP (MOLAP). Although ROLAP and MOLAP may each explore different cube computation techniques.
- The following are general optimization techniques for efficient computation of data cubes.

1. Sorting, hashing, and grouping:

- Sorting, hashing, and grouping operations should be applied to the dimension attributes to reorder and cluster related tuples. In cube computation, aggregation is performed on the tuples (or cells) that share the same set of dimension values.
- To compute total sales by *branch*, *day*, and *item*, for example, it can be more efficient to sort tuples or cells by *branch*, and then by *day*, and then group them according to the *item* name.

- This technique can also be further extended to perform **shared-sorts** (i.e., sharing sorting costs across multiple cuboids when sort-based methods are used), or to perform **shared-partitions** (i.e., sharing the partitioning cost across multiple cuboids when hash-based algorithms are used).

2. Simultaneous aggregation and caching of intermediate results:

- It is efficient to compute higher-level aggregates from previously computed lower-level aggregates, rather than from the base fact table.
- Moreover, simultaneous aggregation from cached intermediate computation results may lead to the reduction of expensive disk input/output (I/O) operations.
- To compute sales by *branch*, for example, we can use the intermediate results derived from the computation of a lower-level cuboid such as sales by *branch* and *day*.
- This technique can be further extended to perform **amortized scans** (i.e., computing as many cuboids as possible at the same time to amortize disk reads).

3. Aggregation from the smallest child when there exist multiple child cuboids:

- When there exist multiple child cuboids, it is usually more efficient to compute the desired parent (i.e., more generalized) cuboid from the smallest, previously computed child cuboid.
- To compute a sales cuboid, C_{branch} , when there exist two previously computed cuboids, $C_{\{branch,year\}}$ and $C_{\{branch,item\}}$, for example, it is obviously more efficient to compute C_{branch} from the former than from the latter if there are many more distinct items than distinct years.

4. The Apriori pruning method can be explored to compute iceberg cubes efficiently:

- The **Apriori property**, in the context of data cubes, states as follows: *If a given cell does not satisfy minimum support, then no descendant of the cell (i.e., more specialized cell) will satisfy minimum support either.*
- This property can be used to substantially reduce the computation of iceberg cubes.

Data cube computation methods

- Data cube computation is an essential task in data warehouse implementation. The pre-computation of all or part of a data cube can greatly reduce the response time and enhance the performance of OLAP, However, such computation is challenging because it may require substantial computation time and storage space.
- This section explores efficient methods for data cube computation which are as follows:
  1. The multi-way array aggregation (Multi-Way) method for computing full cubes.
  2. A method is known as Bottom-Up Computation (BUC), which computes iceberg cubes from the apex cuboid downward.
  3. The Star-Cubing method, which integrates top-down and bottom-up computation.
  4. High dimension OLAP

1. The Multi-Way Array Aggregation (Multi-Way) Method for Computing Full Cubes

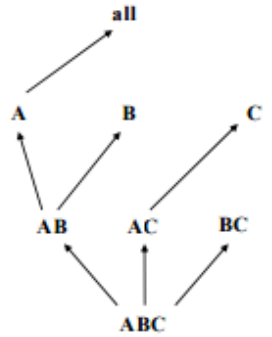
- It is an array-based "bottom-up" algorithm.
- The Multi-way Array Aggregation (or Multi-Way) method computes a full data cube by using a multidimensional array as its basic data structure. It is a typical MOLAP approach that uses direct array addressing, where dimension values are accessed via the position or index of their corresponding array locations.
- Hence, Multi-Way cannot perform any value-based reordering as an optimization technique.

A different approach is developed for the array-based cube construction, as follows:

1. Partition the array into **chunks**. A chunk is a sub-cube that is small enough to fit into the memory available for cube computation.

- Chunking is a method for dividing an n-dimensional array into small n-dimensional chunks, where each chunk is stored as an object on a disk. The chunks are compressed so as to remove wasted space resulting from empty array cells.

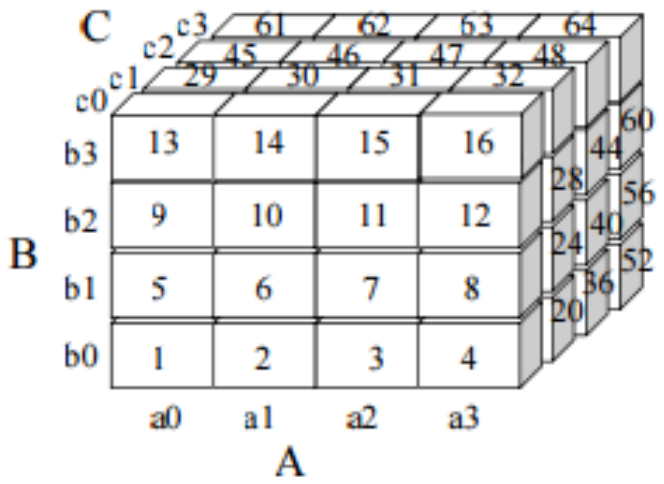
2. Compute aggregates by visiting (i.e., accessing the values at) cube cells: The order in which cells are visited can be optimized so as to minimize the number of times that each cell must be revisited, thereby reducing memory access and storage costs.

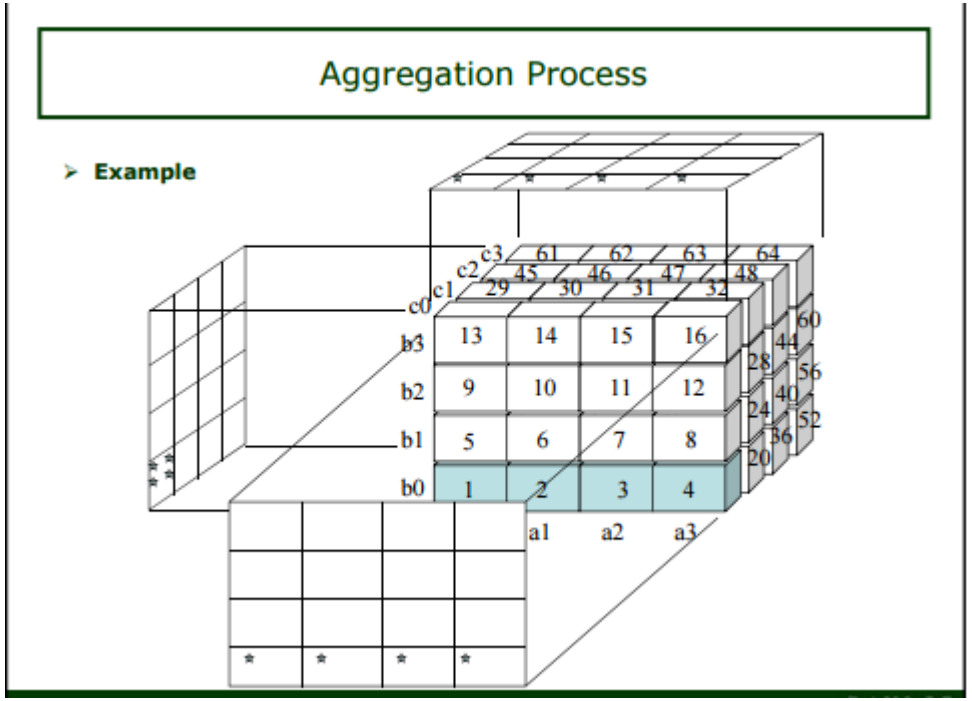


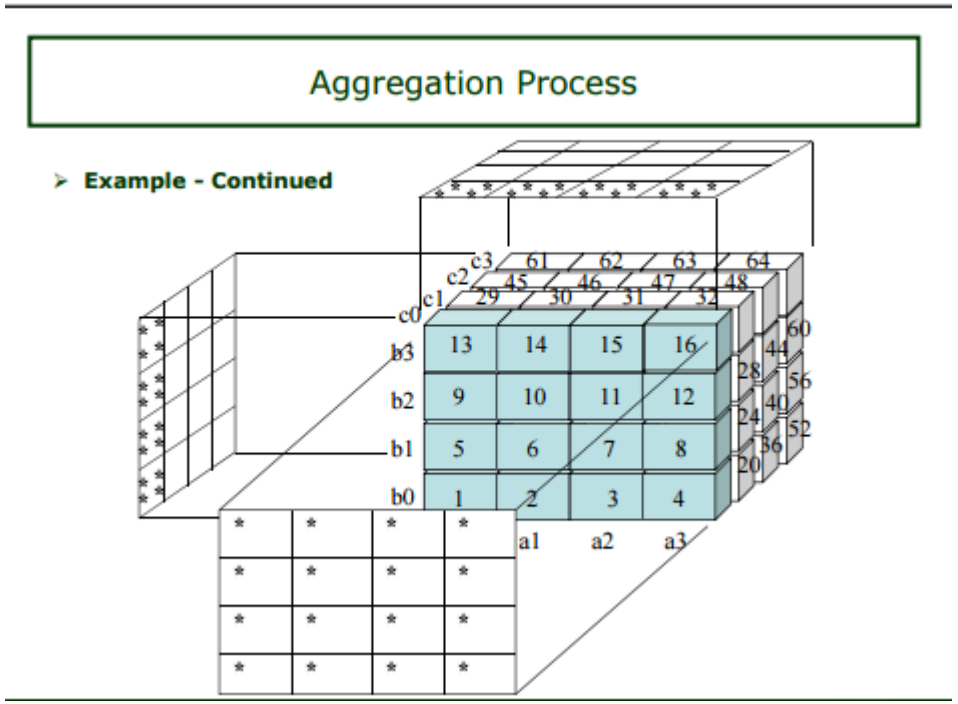
- This chunking technique involves “overlapping” some of the aggregation computations; therefore, it is referred to as multiway array aggregation. It performs **simultaneous aggregation**, that is, it computes aggregations simultaneously on multiple dimensions.

Aggregation Strategy

- Partitions array into chunks
- Chunk: a small sub-cube which fits in memory
- Data addressing
- Uses chunk id and offset
- Multi-way Aggregation
- Computes aggregates in multi-way
- Visits chunks in the order
  - to minimize memory access
  - to minimize memory space

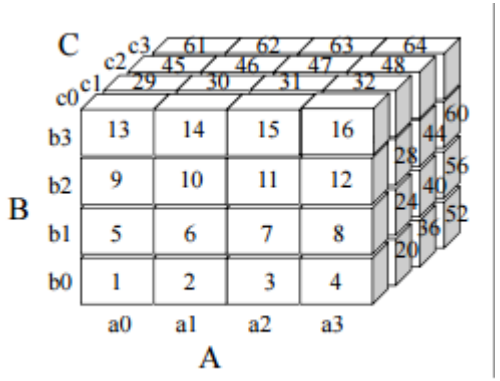






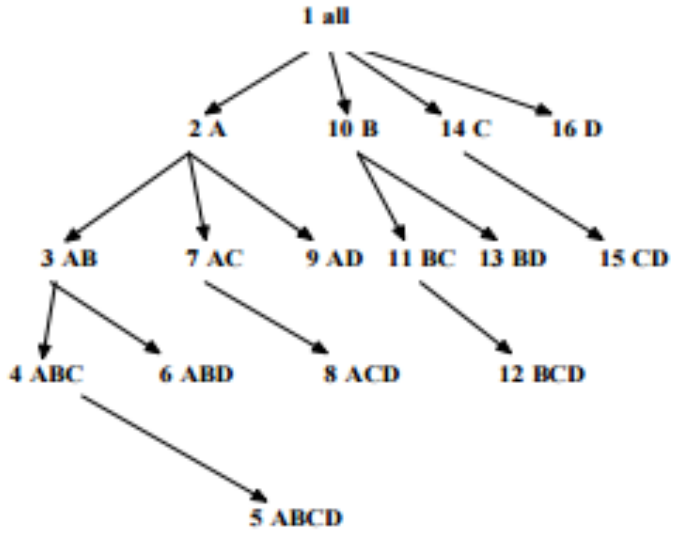
Example:

- Suppose the data size on each dimension A, B and C is 40, 400 and 4000, respectively.
- Minimum memory required when traversing the order, 1,2,3,4,5,…, 64
- Total memory required is 100×1000 + 40×1000 + 40×400



2. Bottom-Up Computation (BUC):

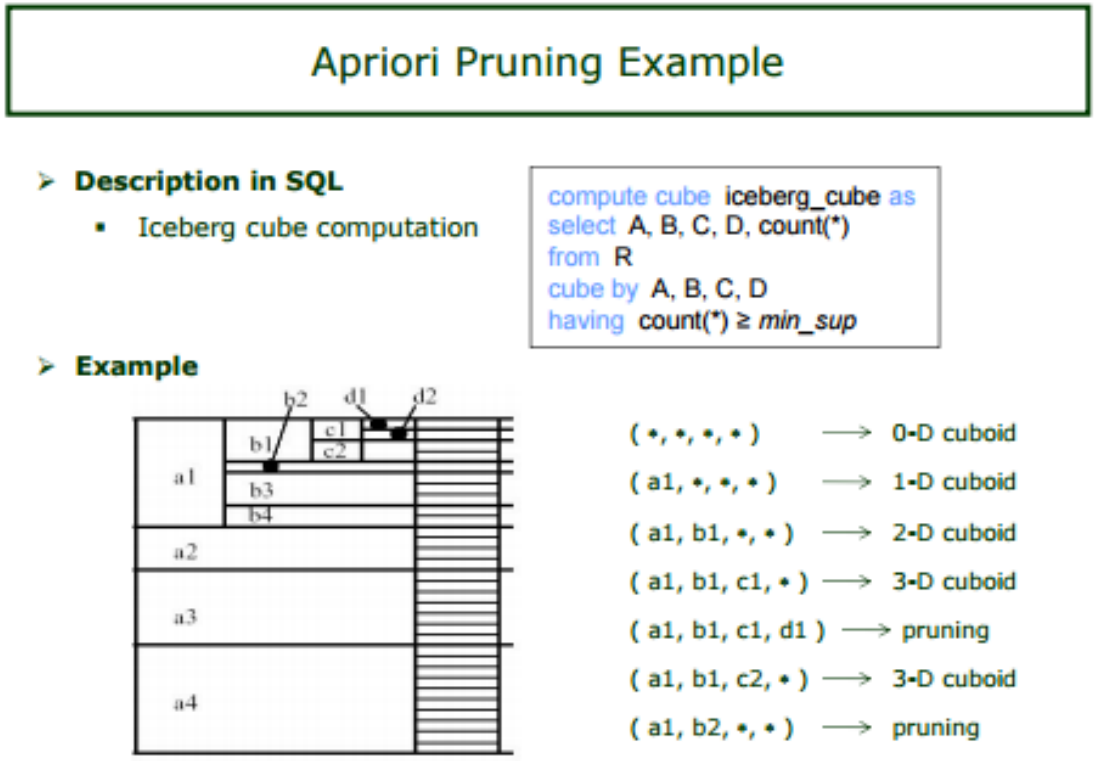
- **Characteristics**
- "Top-down" approach
- Partial materialization (iceberg cube computation)
- Divides dimensions into partitions and facilitates iceberg pruning
- No simultaneous aggregation



- **Iceberg Pruning Process**

Partitioning

- Sorts data values
- Partitions into blocks that fit in memory
- **Apriori Pruning**
- For each block : If it does not satisfy min_sup, its descendants are pruned , If it satisfies min_sup, materialization and a recursive call including the next dimension



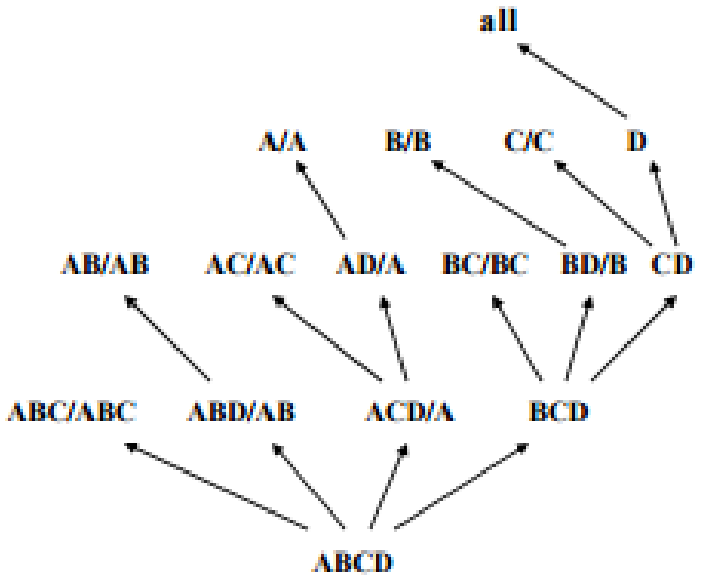
3. Star-Cubing:

Characteristics of Star-Cubing

- Integrated method of “top-down” and “bottom-up” cube computation
- Explores both multidimensional aggregation (as Multi-way) and apriori pruning (as BUC)
- Explores shared dimensions

E.g., dimension A is a shared dimension of ACD and AD

E.g., ABD/AB means ABD has the shared dimension AB

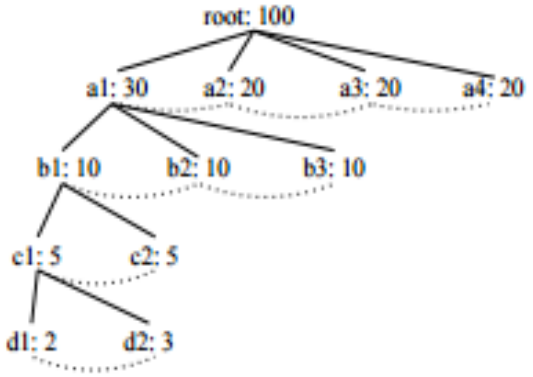


Iceberg Pruning Strategy

- Iceberg Pruning in a Shared Dimension
- If AB is a shared dimension of ABD, then the cuboid AB is computed simultaneously with ABD
- If the aggregate on a shared dimension does not satisfy the iceberg condition (min_sup), then all the cells extended from this shared dimension cannot satisfy the condition either - If we can compute the shared dimensions before the actual cuboid, we can apply Apriori pruning.

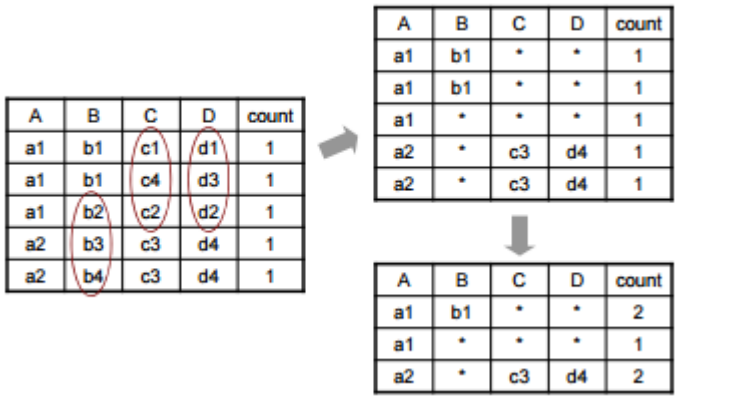
Cuboid Trees

- Tree structure to represent cuboids
- Base cuboid tree, 3-D cuboid trees, 2-D cuboid trees, …
- Each level represents a dimension
- Each node represents an attribute
- Each node includes the attribute value, aggregate value, descendant(s)
- The path from the root to a leaf represents a tuple
- Example • count(a1,b1,,) = 10 • count(a1,b1,c1,*) = 5 → Pruning while aggregating simultaneously on multiple dimensions



Star Nodes

- If the single dimensional aggregate does not satisfy min_sup, no need to consider the node in the iceberg cube computation
- The nodes are replaced by *
- Example (min_sup = 2)



A	B	C	D	count
a1	b1	c1	d1	1
a1	b1	c4	d3	1
a1	b2	c2	d2	1
a2	b3	c3	d4	1
a2	b4	c3	d4	1

A	B	C	D	count
a1	b1	*	*	1
a1	b1	*	*	1
a1	*	*	*	1
a2	*	c3	d4	1
a2	*	c3	d4	1

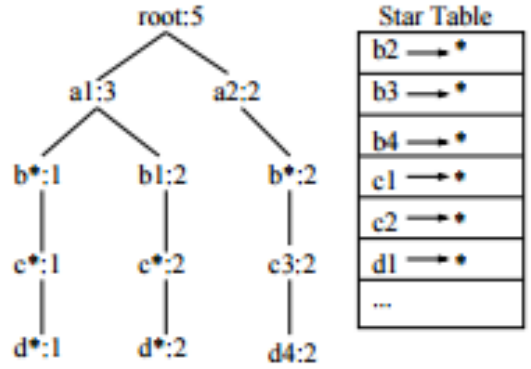
A	B	C	D	count
a1	b1	*	*	2
a1	*	*	*	1
a2	*	c3	d4	2

Star Tree

- A cuboid tree that is compressed using star nodes

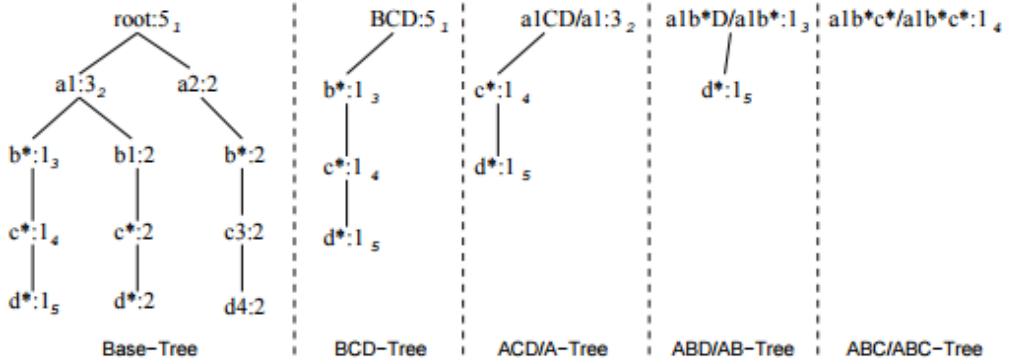
Star Tree Construction

- Uses the compressed table
- Keeps the star table for lookup of star nodes
- Lossless compression from the original cuboid tree



Aggregation

- DFS (depth-first-search) from the root of a star tree
- Creates star trees for the cuboids on the next level

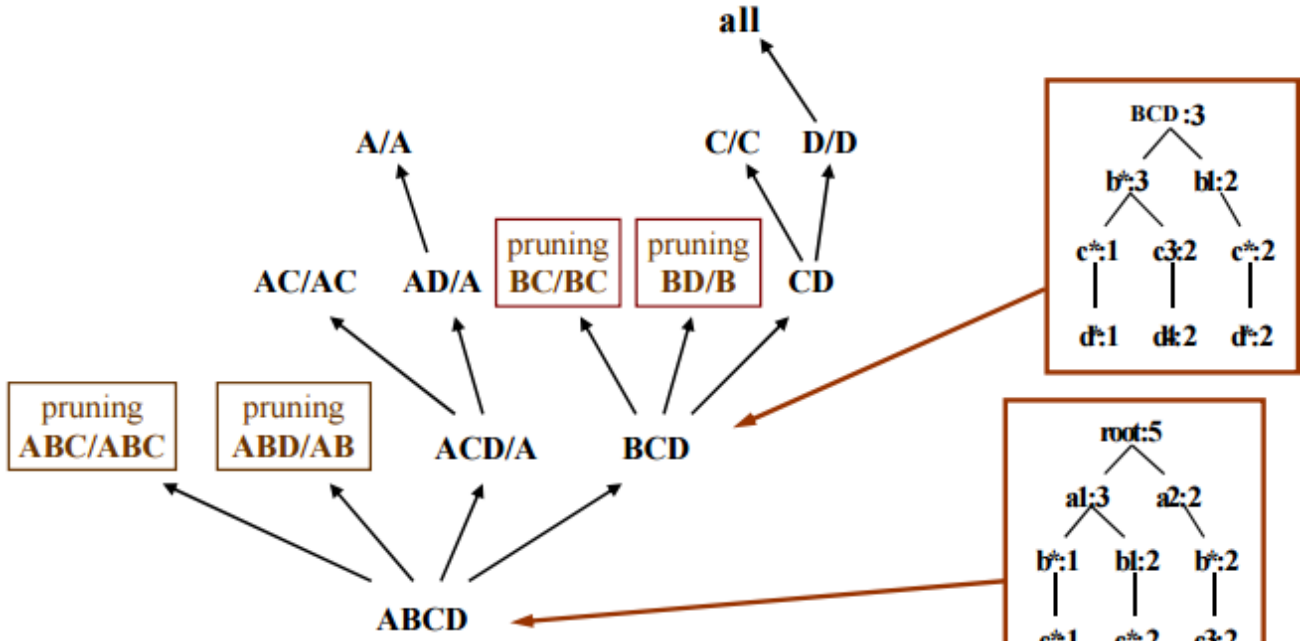


Pruning

- Prunes if the aggregates do not satisfy min_sup
- Prunes if all the nodes in the generated tree are star nodes

*Method**

Multi-way aggregation & iceberg pruning



4. High dimension OLAP:

- High Dimension OLAP(HDOLAP)focuses on handling data cubes with a large number of dimensions efficiently. In high-dimensional scenarios, traditional OLAP techniques can become impractical due to the exponential increase in cube size.
- HDOLAP methods aim to reduce this explosion in cube size by using various techniques like bitmap indexing, dimension reduction, and multi-resolution data representation.
- Example: Let's extend the previous example by adding more dimensions:

  Customer Segment (e.g., Premium, Regular)

  Payment Method (e.g., Credit Card, Cash)

  Promotion (e.g., Discount, Bundle Deal)

  Employee (e.g., Sales Rep)

- With a high number of dimensions, the data cube would become extremely large, making it impractical to compute and analyze using traditional OLAP techniques.
- HDOLAP methods would involve using techniques like bitmap indexing and dimension reduction to efficiently represent the cube.

Benefits of Data Warehousing

- **Better business analytics:** With data warehousing, decision-makers have access to data from multiple sources and no longer have to make decisions based on incomplete information.
- **Faster queries:** Data warehouses are built specifically for fast data retrieval and analysis.
- **Improved data quality:** Before being loaded into the DW, data cleansing cases are created by the system and entered in a worklist for further processing, ensuring data is transformed into a consistent format to support analytics – and decisions – based on high quality, accurate data.
- **Historical insight:** By storing rich historical data, a data warehouse lets decision-makers learn from past trends and challenges, make predictions, and drive continuous business improvement.
- **Data Security:** Data warehousing solutions often include robust security features to protect sensitive and valuable data. Access controls, encryption, and auditing mechanisms help safeguard the data stored in the warehouse.

Trends in Data Warehousing

The field of data warehousing is constantly evolving, with new technologies and trends emerging all the time. Here are some of the key trends in data warehousing.

- **Cloud-based data warehouses:** Cloud-based data warehouses, such as Amazon Redshift, Google BigQuery, and Snowflake, are becoming increasingly popular. These platforms offer a number of advantages over traditional on-premises data warehouses, including scalability, flexibility, and cost-effectiveness.
- **Hybrid data warehouses:** Hybrid data warehouses combine the scalability and flexibility of cloud-based data warehouses with the security and performance of on-premises data warehouses. This approach is ideal for organizations that need to store and process a large volume of data, but also have specific requirements that cannot be met by cloud-based solutions alone.

- **Data lakes**: Data lakes are becoming increasingly popular as a way to store and process unstructured and semi-structured data. Data lakes can be used to store a wide variety of data types, including text, images, video, and audio. Data lakes can be integrated with data warehouses to provide a comprehensive view of an organization's data.
- **Real-time data processing:** Real-time data processing is becoming increasingly important as organizations need to make decisions faster than ever before. Data warehouses are evolving to support real-time data processing, allowing organizations to analyze data as it is generated.
- **Machine learning and artificial intelligence:** Machine learning and artificial intelligence (AI) are being used to automate and optimize various aspects of the data warehousing process, from data ingestion and preparation to modeling and analysis. AI-powered data warehousing tools can help organizations to extract more insights from their data and make better decisions.